

Data-Efficient Language Modeling: From Frontier Advancement to Principle-Guided Model Improvement

Qiushi Engine’s End-to-End Autonomous Research on BabyLM 2026 Strict-Small

Shuxing Yang

Kaihao Zhu

Junjie Yang

Rui Zhao

Junyao Wu

Yize Wang

Wenhao Li

Fujia Chen

Taowen Deng

Shenzhan Hong

Yaqi Li

Zichen Li

Jincheng Mi

Yuang Pan

Hongsheng Chen*

Yihao Yang*

College of Information Science and Electronic Engineering, Zhejiang University

Qiushi Engine Team

*Correspondence: Yihao Yang (yangyihao@zju.edu.cn); Hongsheng Chen (hansomchen@zju.edu.cn)

Abstract

Learning from limited text requires more than repeated exposure: models must learn to use relevant information, apply what they learn to new inputs, and retain useful capabilities during further training. Qiushi Engine conducted a long-horizon, end-to-end autonomous research program on BabyLM 2026 Strict-Small, under limits of ten million corpus words and one hundred million cumulative word presentations. Three complete research stages connected frontier advancement, principle discovery, and principle-guided model improvement, with autonomous literature research, method development, experimentation, analysis, and synthesis throughout. Stage I combined compact restatements, budget reinvestment, and residual incremental learning to build a frontier model. Stage II investigated experience organization, supervision, capability reuse and retention, learning dynamics, and measurement. Exact repetition and aligned restatement produced different patterns of context use, depending on the target relation and prediction window. In controlled tasks, recovering familiar performance did not ensure that unseen inputs could still use a learned computation. These findings support a testable data-efficient learning principle: organize experience around the contextual information and relationships required for prediction; separately design visible information, supervised targets, and functional preservation; and test whether the intended capability is learned, generalizes to new inputs, and survives further training. Stage III applied these findings by retaining source text, masking more local clues, supervising selected targets, and preserving predictions on ordinarily masked inputs. Both continuations from the same parent outperformed ordinary continued training on the complete nine-metric evaluation. Overall advanced from 42.02 to 42.25 across the two model generations, with the second achieving the highest score in the public Strict-Small snapshot of 8 September 2026. Independent studies yielded further findings and methods on compression, relational anchors, shared representations, and measurement controls. Models are available on Hugging Face; code, data construction, evaluations, and research records accompany the GitHub repository. Together, these stages provide a concrete instance of Research RSI—recursive self-improvement of the research process. Scientific understanding, method innovations, and experimental experience from earlier research change subsequent questions and designs; new experiments test and refine them. A frontier advance thus becomes both a better model and a basis for improving how further research is pursued.

Keywords Data-efficient language modeling; BabyLM; context use; capability retention; autonomous research; Research RSI

Contents

1	BabyLM 2026: Learning from Limited Data	4
1.1	The Scientific Problem of Learning from Limited Data	4
1.2	Research Background and Track Rules	5
1.3	Evaluating Language Abilities and Learning Behavior	5
1.3.1	Linguistic Form, Knowledge, and Context Use	6
1.3.2	Transfer Through Downstream Fine-Tuning	6
1.3.3	Reading Behavior and Word Acquisition	7
1.4	Aggregate Scores and the Public Leaderboard	7
1.5	The Main Challenges	9
1.6	Performance Gains and Scientific Contributions	9
1.7	Research Value and Prospective Applications	10
1.8	Three Complete Research Stages Conducted by Qiushi Engine	10
2	Stage I: Frontier Advancement	12
2.1	Choosing Representations, Architectures, and Objectives	12
2.2	Compact Restatements and Budget Reinvestment	12
2.3	Residual Structure and Continued Incremental Learning	13
2.4	Coherent Relations in Late Training	14
2.5	The Experimental Basis for Principle Discovery	15
3	Stage II: Principle Discovery	15
3.1	Relation Type Shapes Context Use	15
3.2	Familiar Performance and Reuse on Unseen Inputs	17
3.3	How Supervision Allocation Changes Relation Learning	19
3.4	Experience Value Depends on Budget, Displacement, and Learning State	20
3.5	A Design Principle and Independent Findings	20
4	Stage III: Principle-Guided Model Improvement	21
4.1	From Mechanism Findings to Training Decisions	21
4.2	Separating Visible Inputs from Supervised Targets	22
4.3	Preserving Functions on Ordinary Inputs	23
4.4	Complete Evaluation Across Two Continuation Seeds	24
4.5	Acquisition and Preservation Measurements	26
4.6	How the Model Tests Refined the Principle	27
5	Research Lineage and Independent Findings	28
5.1	Model Inheritance and the Development of Scientific Questions	28
5.2	Experience Compression, Substitution, and Learning Stage	29
5.3	Relational Anchors, Shared Representations, and Identifiability	30
5.4	Entity Storage, Retrieval, and Semantic Addressing	31
5.5	Optimization, Initialization, and Measurement	32
5.6	Preserving the Full Research Contribution	33

6	Discussion	33
6.1	The Value of Experience and Contextual Dependencies	33
6.2	Coexistence of New Capabilities and Existing Functions	33
6.3	How Qiushi Engine Organizes and Builds on Research	34
6.4	Research RSI: Recursive Self-Improvement of the Research Process	34
6.5	Reusable Contributions and Further Tests	36
7	Conclusion	37
A	Training, measurement, and reproducibility	38
A.1	Computing environment and experimental support	38
A.2	Model versions, teachers, and data sources	38
A.3	Acquisition and preservation algorithm	39
A.4	Evaluation aggregation, randomness, and trajectories	41
B	Complete local evaluation vectors	42
C	Additional controls and mechanism results	43
C.1	Downstream fine-tuning randomness	43
C.2	Target deletion and source isolation	43
C.3	Measurement populations and subsequent corrections	44
D	Public leaderboard metrics	45
E	Research repository and use of materials	45
F	Research topics and findings	48

1 BabyLM 2026: Learning from Limited Data

Large-scale training has shown how far language-model capabilities can grow with additional resources. Learning from limited text asks a complementary question: how much more can a model learn from the experience already available to it? A better method must do more than fit that text. It should help the model use relevant information, apply what it learns to new expressions, and retain useful capabilities as training continues.

BabyLM provides shared experimental conditions for this question. This report concerns its Strict-Small track and the complete research program conducted by Qiushi Engine: building a frontier model, investigating the conditions behind its behavior, and using those findings to improve the next model. Model scores are one outcome. The resulting methods and testable explanations are another.

1.1 The Scientific Problem of Learning from Limited Data

Language models learn by predicting text. Training compares a model’s predicted probabilities with the words that actually occur, measures the error through a loss, and adjusts the parameters. In this report, *supervision* specifies which predictions contribute to learning: a target is a token to be recovered, and a target position identifies where that token occurs. Repeated training can establish lexical, syntactic, semantic, and contextual regularities. The scientific question is which regularities actually support a correct answer.

Suppose a passage states that a key was moved from a drawer to a box, and a later query asks where the key is now. The required computation uses the change of state. A model that instead relies on the familiar association between “key” and “drawer” may fit many ordinary sentences without learning that operation. If nearby words usually reveal the target, training offers little incentive to consult earlier information. **Encountering relevant information and learning to use it are distinct achievements.**

This distinction matters especially when data are scarce. Adding a new passage, repeating an old one, and restating its content in different words all consume training budget, but need not provide interchangeable learning opportunities. Repetition can consolidate a pattern; re-statement can vary its expression; a new source can broaden coverage. Their value depends on the model’s current capabilities and remaining training budget. Data-constrained scaling studies likewise examine repeated exposure jointly with the amount of nonrepeated data and the training configuration [1].

Data-efficient language modeling studies how to use limited learning material more effectively. Parameter count, corpus size, cumulative exposure, and compute measure different resources: capacity, available text, repeated presentations, and execution cost. A small model can read a large amount of text repeatedly; a larger model can have limited experience. BabyLM’s strict English tracks constrain both the corpus and cumulative exposure, allowing methods to be compared under common data limits.

Our central question is: **what training experience teaches a model to use information relationships in context, and what allows that ability to remain usable during further learning?** We use *contextual dependencies* to mean the contextual information and relationships required to make a prediction. This question connects language acquisition, compositional generalization—using learned elements in new combinations—and continual learning—acquiring new capabilities while retaining previous ones. Natural text, controlled relational tasks, and full language-model training examine different parts of this question.

1.2 Research Background and Track Rules

Launched in 2023, BabyLM enters its fourth edition in 2026 as a shared task and workshop associated with EMNLP 2026. It brings together natural language processing, computational cognitive science, and language-acquisition research through training data, evaluation software, baseline models, and public comparisons [2, 3]. Earlier editions have established a body of work on data processing, model design, and learning behavior [4].

Its value lies in the experimental question as much as the leaderboard. For machine learning, BabyLM provides a tractable setting for comparing training methods. For language and cognition, it provides models whose input and learning trajectories can be manipulated. For research teams with limited resources, it makes training from scratch and retaining intermediate models more feasible. The task does not require a complete simulation of a child; it makes learning from restricted input a shared object of investigation.

The 2026 tracks include English Strict-Small, English Strict, and Multilingual, covering English, Dutch, and Chinese. Results in this report belong only to Strict-Small. The two English budget levels are summarized in Table 1 [5].

Table 1 Data limits for the English tracks. Word counts follow the track rules, rather than the model tokenizer. M denotes million and B denotes billion.

Track	Corpus limit	Cumulative exposure limit
Strict-Small (this study)	10M words	100M words
Strict	100M words	1B words

The corpus is the collection available for training; cumulative exposure counts the text actually presented during training. Reading a thousand-word article ten times still uses one article but consumes ten thousand words of exposure. Rewritten text, repeated passages, and additional inputs used to preserve existing behavior must also be accounted for under the applicable rules. A tokenizer converts text into computational units called tokens. Because one word can produce several tokens, tokenizer counts cannot replace the track’s word counts.

Intermediate checkpoints—model weights saved at successive training amounts—are required for evaluating learning trajectories. External teachers and auxiliary training methods have additional rules; satisfying the final word count alone is insufficient to document them. Appendix A records the data, teacher roles, and training settings used here. The evaluation therefore concerns both the final score and the experience through which it was obtained.

A fixed budget still leaves substantial scientific freedom. Data content, expression, vocabulary, architecture, objectives, learning rate, batch size, presentation order, and model selection all remain consequential choices. The learning rate controls the magnitude of parameter updates; the batch size controls how much evidence is combined in an update. At a fixed exposure budget, larger batches can also mean fewer updates. Strict-Small supplies common constraints, not a prescribed training recipe.

1.3 Evaluating Language Abilities and Learning Behavior

No single test establishes grammatical knowledge, state tracking, commonsense use, transfer to new tasks, and correspondence with human language behavior. BabyLM’s nine top-level metrics measure these separately. **The overall score combines task performance with measures**

of human behavioral correspondence. Each component must be interpreted through its own task, scale, and reference level.

Table 2 The nine top-level English evaluation metrics. The first seven form NLP Average; the final two form Human-like Average. Scores use the fixed official-compatible evaluation implementation.

Metric	Main question	Interpretation
BLiMP	Does the model prefer the grammatical member of a minimal pair?	Choice accuracy across grammatical phenomena.
BLiMP Supplement	Does it handle additional linguistic phenomena, including question answering and reference?	Aggregation of supplementary task groups.
EWoK	Does contextual information support elementary world knowledge?	Relative plausibility of continuations under different contexts.
Entity Tracking	Can it recover an object’s state after moves or updates?	Candidate-answer accuracy after state changes.
COMPS	Can properties transfer to new concepts despite distractors?	Property, inheritance, and interference tests.
GlobalPIQA	Can it judge everyday physical actions and uses?	Choice accuracy across subsets with different formats.
(Super)GLUE	Do pretrained representations support downstream understanding?	Mean task metrics after prescribed fine-tuning.
Reading	Do model predictions explain variation in human reading?	Explanatory gain beyond word length, frequency, and other controls.
AoA	Does the learning order of words correspond to children’s acquisition?	A trajectory-based correlation measure.

1.3.1 Linguistic Form, Knowledge, and Context Use

BLiMP isolates small grammatical contrasts such as subject–verb agreement. EWoK changes the context and asks whether the model’s judgment changes accordingly. COMPS tests whether a new concept inherits the appropriate property and whether unrelated concepts interfere. Entity Tracking requires the model to recover a state after a sequence of operations [6–9]. These tasks separate plausible language from the use of the information needed for a particular answer.

“The keys are on the table” illustrates agreement; a key moved from a drawer into a box illustrates state updating; assigning a newly named animal to a category illustrates property inheritance. These are explanatory examples, not benchmark items. They show why the challenge extends beyond remembering facts: limited experience must support different structural and relational operations.

Most of these tests score sentences or candidate answers with the pretrained model, without training on the individual questions. This is zero-shot evaluation. Chance performance depends on the task format. The GlobalPIQA English evaluation used here has 103 four-choice items and 100 two-choice items, with equal weight assigned to the two subsets; uniform random choice therefore has an expected score of 37.5. A score near 40 cannot be interpreted without that reference [10, 11].

1.3.2 Transfer Through Downstream Fine-Tuning

(Super)GLUE asks whether pretrained representations support language understanding after task-specific training, or fine-tuning. BabyLM uses seven selected tasks covering yes/no ques-

tions, entailment, paraphrase, multi-sentence reading judgments, and reference resolution [12, 13]. It tests whether pretraining supplies a useful basis for subsequent learning.

Some tasks use accuracy and others F1. Accuracy measures the proportion of correct answers; F1 combines precision and recall for the relevant predictions. The score is consequently neither the full original GLUE/SuperGLUE suite nor a pooled accuracy over all questions. Fine-tuning seeds, classification heads, training settings, and model selection must be kept consistent when comparing models.

1.3.3 Reading Behavior and Word Acquisition

Reading measures how model expectations relate to human reading, rather than accuracy on reading-comprehension questions. A less probable word has higher *surprisal*, defined as its negative log probability given the preceding context. The evaluation asks whether surprisal explains eye movements or word-by-word reading times beyond factors such as length and frequency [14].

The score follows the BabyLM 2026 evaluation implementation [11]. If control variables explain a proportion R_0^2 of the variance and adding model predictions raises this to R_1^2 , the normalized gain is

$$q = \frac{R_1^2 - R_0^2}{1 - R_0^2}. \quad (1)$$

An increase from 20% to 24% explained variance gives $0.04/0.80 = 0.05$, or 5 on a percentage scale. Small Reading values can therefore represent meaningful explanatory gains. The leaderboard aggregates the prescribed reading responses [15].

AoA denotes Age of Acquisition. The evaluation estimates word-learning progress from prediction curves across checkpoints and compares it with children’s acquisition order [16]. The implementation used here computes Pearson correlation, returning zero when fewer than three words have valid estimates or the correlation test gives $p > 0.1$; a negative correlation that passes this threshold remains negative [11]. Appendix A.4 specifies the scoring version and checkpoint trajectories. **An AoA score of zero does not mean that the model learned no words.** Reading and AoA make behavioral correspondence an explicit dimension alongside task competence.

1.4 Aggregate Scores and the Public Leaderboard

Let $m_1, \dots, m_9$ be the component scores, N the mean of the seven language-task scores, and H the mean of the two human-behavior scores. Overall Average is

$$O = \frac{1}{9} \sum_{j=1}^9 m_j = \frac{7N + 2H}{9}. \quad (2)$$

Each component has equal weight; NLP and Human-like therefore have unequal total weights. Overall permits comparison under a shared protocol. It is not the percentage of language a model understands, and it cannot be compared directly with large-model scores from different tests and training budgets.

The two representative models are **Qiushi-Engine-Frontier-Advancement** and **Qiushi-Engine-Principle-Guided-Frontier-Advancement**, abbreviated *Frontier* and *Principle-guided*. Their public Overall scores are 42.02 and 42.25. The latter is the highest Overall in the retained Strict-Small snapshot.

Table 3 includes these two models and the eight highest-Overall entries from other publishers, without the team’s historical submissions or an adjusted ranking column. The snapshot was recorded on 8 September 2026 at 20:53 Beijing time. This is a comparison of public submissions, distinct from the determination of workshop awards [15].

Table 3 Two representative models and eight external public submissions. Readable names are used here; full identities and links accompany the data. Scores retain the leaderboard’s displayed precision.

Publisher/model	Overall	NLP	Human-like
Qiushi Engine / Principle-guided	42.25	53.15	4.10
Qiushi Engine / Frontier	42.02	52.86	4.08
go76dof / WWM curriculum	41.80	52.97	2.71
Serdar404 / RecGPT-10M	41.53	52.40	3.46
Wector1 / Wordpiece-24-4	41.31	52.85	0.92
gyaudhw / compagg relay	40.94	51.81	2.92
gyaudhw / compagg relay (s0)	40.93	51.79	2.92
svsatheesh / BabySteps	40.86	53.59	-3.72
qyxu1994 / instanton-hybrid	40.78	51.53	3.18
gyaudhw / synonly (s0)	40.67	51.46	2.91

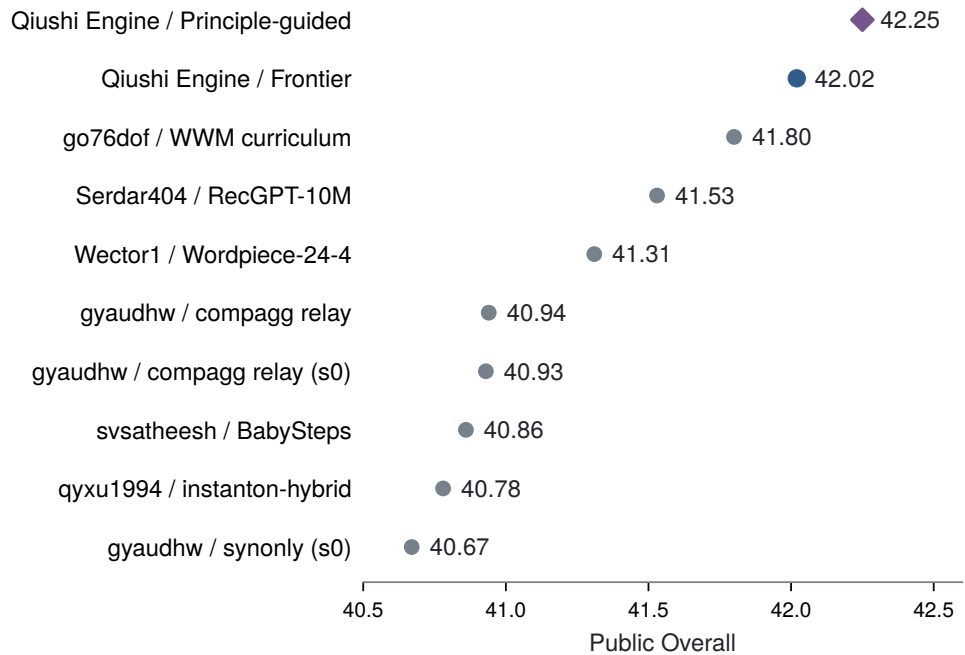


Figure 1 Public Overall scores on BabyLM 2026 Strict-Small, 8 September 2026. Points and labels show the displayed scores for the two Qiushi Engine models and the eight external submissions in Table 3.

The external entries lie between 40.67 and 41.80, a span of 1.13 points, with adjacent differences as small as 0.01. The second-generation model exceeds the highest external displayed score by 0.45. Closely spaced totals make component-level analysis important: which abilities improved, which incurred costs, and whether repeated training from the same starting model

retains the advantage. The leaderboard establishes the public comparison; common-parent experiments below test the training method itself.

Similar totals also conceal different capability profiles. The highest NLP score in the comparison is 53.59, yet weaker human-behavior scores prevent that model from achieving the highest Overall. BLiMP ranges from 67.13 to 73.11, Entity Tracking from 16.59 to 29.40, and GlobalPIQA from 36.08 to 40.68, near its 37.5 chance reference. Syntactic preferences, changing states, and physical commonsense remain unevenly learned. Appendix D retains all nine components.

1.5 The Main Challenges

Coverage versus practice. A fixed word budget can buy broader content or more rehearsal. Compression may create room for additional examples while removing relational clues. Rewriting may provide useful variation or redundancy. Data quality must be judged by what the transformation teaches, not text volume or fluency alone.

Correct prediction versus relevant information use. A model may complete a target using nearby collocations without reading the source. Loss reduction can be genuine even when the desired relational ability has not formed. Keeping targets fixed while changing visible clues, or keeping text fixed while changing supervised positions, separates these possibilities.

New learning versus retention. Training a particular relation can improve that task while degrading ordinary language abilities. Familiar-item performance may recover even when new names cannot use the learned computation. An integrated improvement requires measuring acquisition and retention together.

Interactions among representation, architecture, and optimization. Tokenization determines the input units; architecture determines the available computations; objectives and optimization determine which relationships are practiced. Early rankings can reverse, and the same random seed does not ensure matching initial parameters across architectures. Useful controls isolate the differences relevant to the proposed explanation.

Training a model versus conducting the research. The cost of one final training run is not the cost of finding its method. Candidate selection, mechanism experiments, full evaluation, and error analysis are additional work. Autonomous research must connect these activities over time: preserve reliable starting points, identify informative questions, implement controls, correct mistakes, revise explanations, and construct another executable method. This complete process is the relevant setting for assessing Qiushi Engine’s research contribution.

1.6 Performance Gains and Scientific Contributions

Because Overall averages nine metrics, a 0.9-point increase in one component with no other change yields a 0.1-point increase in Overall. Several smaller changes can yield the same total. A narrow aggregate margin may therefore reflect identifiable behavioral changes, but sample size, training randomness, and model selection also matter. Component results and matched comparisons establish what the margin means.

The study compares the new methods with ordinary continuation from the same first-generation model. Ordinary continuation achieves 42.0926 and 42.1159 across two continuation seeds. Dense masking with sparse supervision reaches 42.2025 and 42.1789 at the same cumulative word exposure. Adding ordinary-input preservation yields 42.2464 and 42.2317, with extra preservation presentations accounted for separately.

The scientific contribution extends beyond these differences. Stage I produced a way to compress expression and reinvest the word budget, an extensible model structure, and a complete training and evaluation implementation. Stage II established relation-dependent context-use effects and separated familiar performance from reuse on unseen inputs. Stage III converted those findings into training operations and tested their practical value. **New observations, discriminating explanations, executable methods, and evidence together define the contribution.**

Independent investigations studied how sparse labels identify relational structure, how new names become associated with known objects, and how explicit memory retrieves the appropriate state. These yielded work on relational anchors, identity matching, and entity memory, developed in Section 5.

1.7 Research Value and Prospective Applications

Data-limited research produces methods that others can inspect and adapt. Specialized domains and low-resource languages often lack sufficient high-quality training text. Preserving relevant content, organizing corresponding expressions, selecting supervision, and continuing learning are therefore plausible transfer directions. The present experiments provide concrete methods and hypotheses for those applications, rather than an already established cross-domain result.

Small-scale experiments also make mechanisms easier to study. Data and training variables can be controlled directly, and changes to a method can be tested economically. Local prediction shortcuts, supervision imbalance, and excessive preservation constraints suggest specific questions for larger models: when do the same conditions recur, and how do they vary with scale, task, and budget?

For language and cognitive science, the model is an experimental object whose experience can be manipulated and whose learning can be followed. Joint evaluation of grammar, conceptual inheritance, reading behavior, and acquisition order helps identify when better task performance coincides with more human-like behavior and when the two diverge.

For autonomous AI research, the contribution is a sustained scientific process. Qiushi Engine turns differences in trained-model behavior into questions, constructs comparisons, revises explanations, and uses the resulting understanding in subsequent model design. Its models, methods, and scientific judgments develop together. We describe the reuse of this accumulation in subsequent research as Research RSI—recursive self-improvement of the research process. Section 6.4 develops its concrete realization, relationship to prior work, and empirical scope.

1.8 Three Complete Research Stages Conducted by Qiushi Engine

Qiushi Engine conducted the research across all three stages: data preparation, representation and architecture exploration, objective design, experimental implementation, training, evaluation, mechanism analysis, and synthesis of the findings. The team supplied goals, resources, and stage-level requirements; Qiushi Engine carried out the scientific work within that setting. Computational research used the AI Lab skill and associated scientific programs on two NVIDIA H100 GPUs and CPUs. Section 6.3 describes the research organization; Appendix A.1 records the compute environment.

Each stage is a complete research process, including questions, hypotheses, method construction, experiments, analysis, and reporting. The stages differ in their scientific aims, while data, models, experimental methods, and findings connect them.

Stage I: frontier advancement. Qiushi Engine explored text construction, tokenization, representation, architecture, optimization, and training schedules under limited data. Compact restatements, budget reinvestment, and residual incremental learning became a combined method that produced the first frontier model. Its training also exposed changing method rankings and differences between local learning and broad performance.

Stage II: principle discovery. Qiushi Engine treated those models and observations as research objects. It proposed competing explanations and tested them through interventions on data, windows, supervision, representations, and internal computations. The work covered experience structure and compression, inductive biases, learning signals, retention, curricula, learning dynamics, and measurement. Relation learning and capability reuse informed the next model; anchors, identity matching, initialization controls, and measurement methods also produced independent findings.

Stage III: principle-guided model improvement. Using the first model and its existing text, Qiushi Engine changed the visible input, supervised positions, and preservation constraints. Ordinary continuation, target-set comparisons, mechanism tests, and full evaluation established the second model and refined earlier explanations of supervision quantity and preservation strength. Figure 2 summarizes the continuity of the three research processes.

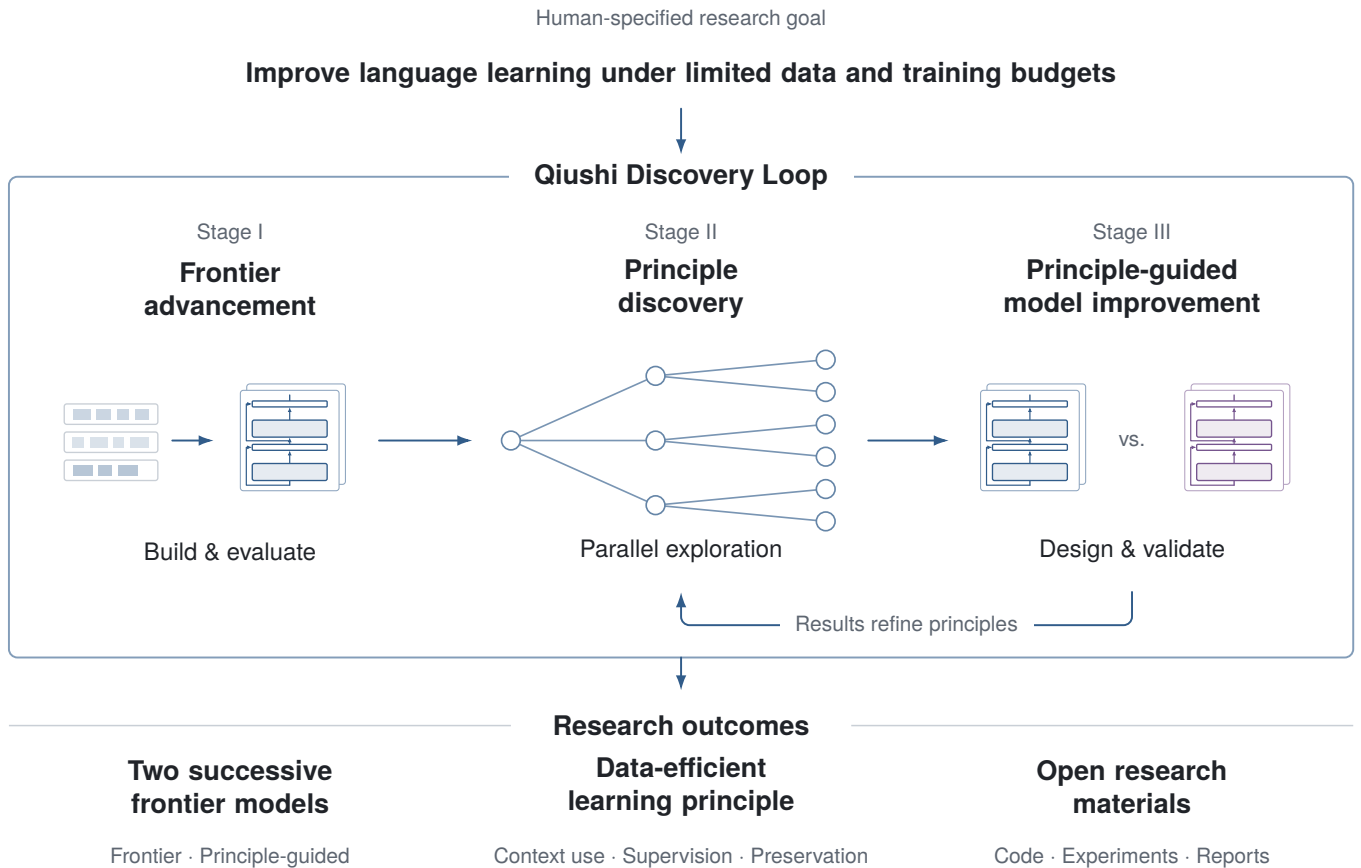


Figure 2 Qiushi Engine’s three-stage autonomous research program. Frontier advancement, principle discovery, and principle-guided model improvement each comprise a complete research process. Models, methods, and scientific understanding develop across the stages, producing two successive frontier models, data-efficient learning principles, and open research materials.

The resulting design principle is: **organize limited experience around the contextual dependencies a capability requires; make relevant information available during prediction, train the model to use it, and evaluate acquisition, reuse, and retention together.** Acquisition asks whether the target behavior is learned; reuse asks whether new objects or expressions can use it; retention asks whether it remains available after further learning. The practical variables are visible information, supervision targets, and preservation conditions. The principle developed through model construction and mechanism experiments before informing the third-stage design.

2 Stage I: Frontier Advancement

Building a frontier model under strict data limits required coordinated choices about text, architecture, and training. Qiushi Engine compared data processing and model designs, then investigated how to improve a mature model as its exposure budget approached the limit. Complete evaluation determined which combinations succeeded. Beyond the resulting model, this work established the experimental basis for continued training and mechanism analysis.

2.1 Choosing Representations, Architectures, and Objectives

Text representation determines the units a model learns from. Character representations produce longer sequences of finer units; subword representations combine frequent fragments and require embeddings for a larger vocabulary. Autoregressive models predict subsequent text from preceding text, while masked language models recover hidden tokens from visible context. Qiushi Engine compared these choices along with depth, width, vocabulary, and training schedules.

Other candidates used sparse computation, auxiliary memory, or additional word-form features. Lower loss on one objective was informative but insufficient: a configuration still had to improve the relevant language abilities under comparable budgets. Architecture comparisons therefore required shared-parameter checks; tokenizer comparisons required revised sequence and exposure accounting; objective comparisons required measuring both gains and costs. The investigation moved from local component effects to a training design whose parts worked together.

The final route combined compact restatements, residual learning, and evaluation-based selection of a late-training increment. Other architectural studies retain their independent results in Section 5 and Appendix F. The following sections describe the actual formation of the released model.

2.2 Compact Restatements and Budget Reinvestment

Original passages are paired with corresponding rewrites. An original passage is the *source*; an alternative expression of its content is the *restatement*. Experimental records also use *text view* and *aligned paraphrase* for this second expression. A source–restatement pair creates a prediction problem different from exact repetition, but both expressions consume words. Compact restatements reduce redundancy while retaining their connection to the source, freeing budget for additional material.

Shortening each restatement released words for additional passage pairs. The core collection grew from 10,094 to 12,155 pairs, an increase of 2,061 or 20.42%, within a fixed-size corpus subset called the replacement block. Its 423,520 words comprise 261,803 source words, 161,708

restatement words, and nine padding words. The pairs came from 4,529 source documents, each of which could supply multiple passages.

Table 4 Compact restatements expand paired experience within a fixed word budget. The replacement block remains at 423,520 words.

Quantity	Count	Interpretation
Original core pairs	10,094	Initial paired experience
Additional pairs	2,061	Reinvestment of saved words
Final pairs	12,155	20.42% more than the original set
Source documents	4,529	May contribute multiple pairs
Source/restatement words	261,803/161,708	Both expressions retained
Replacement-block words	423,520	Includes nine padding words

Compression and reinvestment change both expression and coverage: the restatement becomes shorter, and additional pairs broaden the available source information. Distinguishing these effects subsequently motivated fixed-budget substitution and target-deletion studies.

Training revealed a reversal in relative performance. A seven-metric screening average combined BLiMP, Supplement, EWoK, Entity Tracking, COMPS, GlobalPIQA, and Reading. Because it includes Reading and excludes (Super)GLUE, it is not the official NLP Average. In the corresponding matched comparison, the compact-restatement method was about 0.83 points below its reference at 20M cumulative words, but 1.29 and 1.35 points above at 70M and 80M. An early ranking would therefore have missed the benefit that emerged later, motivating study of data value across learning trajectories.

The stage also investigated continuous window packing, which organizes adjacent text into sequences the model can process. These studies produced methods for word-budget use, truncation, and context organization. Only the adopted data and weights entered the final model’s lineage; the remaining implementations are retained as separate research outputs.

2.3 Residual Structure and Continued Incremental Learning

The representative model uses a DeBERTa-v2-style masked language architecture [17, 18]: eight layers, hidden width 480, eight attention heads, and a vocabulary of 16,384. Hidden width is the size of the internal vector at each position; attention heads extract information from different contextual positions. Pretraining uses a 256-token prediction window. A byte-level BPE tokenizer, trained on the restricted corpus, merges frequent byte sequences into vocabulary units. The ordinary objective uses 15% whole-word masking, masking a selected word’s subword tokens together.

A residual branch adds a learned correction to an existing representation. Here the branch projects width 480 down to 128 and then back to the original width. During joint backbone training, its form is

$$h' = h + s U \sigma(D \text{ Norm}(h)), \quad (3)$$

where D and U reduce and restore dimensionality, σ is a nonlinear transformation, Norm normalizes the representation, and s scales the correction. The upward projection is initialized to zero, so the new branch initially leaves the representation unchanged. Joint training uses $s = 1.75$, with the relevant parameters learned together from random initialization. The construction draws on residual and parameter-efficient learning [19, 20]; here it is combined with budget-constrained data organization, training from scratch, and later incremental learning.

At 82,012,495 words of cumulative exposure, the jointly trained model has 35,463,008 parameters. Qiushi Engine then froze those parameters and added a dedicated trainable increment containing 995,584 parameters in 48 tensors. Total model size became 36,458,592 parameters. This stage used 101 updates and 3,992,800 additional words, bringing cumulative exposure to 86,005,295.

Table 5 The two training phases in the first-generation model’s lineage. Trainable parameter counts and computational savings are separate quantities.

Phase	Updated parameters	Total parameters	Cumulative words
Joint backbone training	Backbone and residual branches	35,463,008	82,012,495
Frozen-base incremental training	New branch only	36,458,592	86,005,295

Incremental training used both ordinary prediction loss and a KL preservation term. Kullback–Leibler divergence measures the difference between predictive probability distributions; the preservation loss constrains changes to the previous model’s predictions. Thus, the first generation already used output preservation. The subsequent question was more specific: under which inputs should which functions be preserved, and how should this constraint coexist with new learning?

The dedicated increment provides a useful control. Disabling it restores the frozen base’s output; enabling it adds the learned correction. Retention is evaluated with the branch enabled. Only about 2.73% of parameters are trainable. Freezing the base removes its parameter updates, but gradients must still pass through intervening frozen operations to train incremental modules at earlier layers. Computation also includes reference-model predictions. The trainable parameter fraction is therefore not a compute-saving estimate.

2.4 Coherent Relations in Late Training

Freezing a common base permits direct comparisons of text organization. A key control held the starting model, increment, exposure, and update count fixed while shuffling coherent segments within each training row. At residual scale 1.0, coherent training achieved 44.1064 on the seven-metric average, compared with 43.1214 after segment shuffling: a difference of approximately 0.9850. The same words produced different outcomes when their coherent organization changed.

Table 6 Late-training references and controls. The screening average comprises BLiMP, Supplement, EWoK, Entity, COMPS, GlobalPIQA, and Reading. The final two rows are the closest organization-only comparison; ordinary and incremental continuation also differ in updated parameters, optimizer, and objectives.

Condition	Cumulative words	Seven-metric mean
Backbone before continuation	82,012,495	43.9594
Ordinary joint continuation	86,006,729	43.7707
Increment with mismatched correspondence	86,005,413	44.0129
Coherent increment, scale 1.0	86,005,295	44.1064
Increment with within-row segment shuffling	86,005,295	43.1214

After training the increment at scale 1.0, evaluation of alternative inference scales selected 0.75 for release: three quarters of the learned branch output is added to the original represen-

tation. This was an evaluation-based model-selection decision. The final first-generation local Overall was 42.0240, displayed publicly as 42.02.

Individual answers and local capabilities also changed as aggregate performance improved. These observations led directly to the next questions: how does incremental learning balance new functions with established abilities, and which inputs can still use the computation that has been learned?

2.5 The Experimental Basis for Principle Discovery

Stage I produced a model that could be trained further, reusable source–restatement data, and phenomena requiring explanation. Coherent organization outperformed segment shuffling; compact–restatement benefits changed over training; local improvements did not always yield aggregate gains. Qiushi Engine used these observations to move from configuration selection to mechanism studies of learned relations, reuse on new inputs, and retention during further learning.

3 Stage II: Principle Discovery

Stage I showed that changing text organization and training could improve a model. In Stage II, Qiushi Engine turned those practical results into questions about experience, representation, objectives, architecture, reusable capabilities, learning dynamics, and measurement. It proposed explanations, designed controls that could distinguish them, conducted experiments, and revised its understanding. The outputs included both principles relevant to the next model and independent theoretical constructions, mechanism studies, and experimental methods.

Two complementary investigations connect experience to capability: natural-text experiments establish how training changes context use, while controlled tasks test whether new inputs retain access to learned computations after further training. Their findings inform the next training design. Section 5 develops the other research branches, and Appendix F records their methods and evidential status.

3.1 Relation Type Shapes Context Use

Exact repetition presents the same expression again. Aligned restatement presents corresponding content in different words. Both increase exposure to related text, but impose different prediction problems. If they supplied interchangeable information exposure, comparable budgets should produce similar source-use behavior. If the model instead practices particular source–target relations, transfer should depend on the relation required by the later target.

Consider a source that describes an object’s location and a restatement that describes the same location in another sentence form. Predicting the masked location could require reading the source or merely recognizing a local collocation. Replacing the source with length-matched unrelated text while leaving the target unchanged measures the contribution of that source. We use *contextual dependency* to mean the information relationship needed for the target prediction.

All conditions inherit the existing rewritten corpus. The reference adds none of the compact pairs under investigation; two other conditions add exact repetition or aligned restatement. These test the relation in the additional experience. Each pairing condition also has same-window and split-window variants. The latter preserve that condition’s text but place the source and corresponding passage in separate windows, so a prediction cannot access both.

Thus, relation-type comparisons and within-condition, text-matched window comparisons test different variables.

Source use is measured with fixed target tokens under true-source, unrelated-source, and length-matched neutral-context inputs. For a correct token with probability p , negative log-likelihood (NLL) is $-\log p$: smaller values mean better predictions. Averaging over the specified targets gives cross-entropy (CE); natural logarithms express the result in nats. Let T, U, N denote the respective mean losses. Define

$$A_T = N - T, \quad A_U = N - U. \quad (4)$$

Larger A_T means that the true source helps more than the neutral context. A_U tests whether unrelated source material produces a similar change. A substantial improvement in A_T with little change in A_U supports an effect specific to the relevant source content.

Targets are also grouped by whether their exact tokenizer ID occurs in the source. This measures lexical token overlap, not whether a whole word, fact, or meaning is new. Figure 3 uses nonoverlapping target tokens in compact restatements.

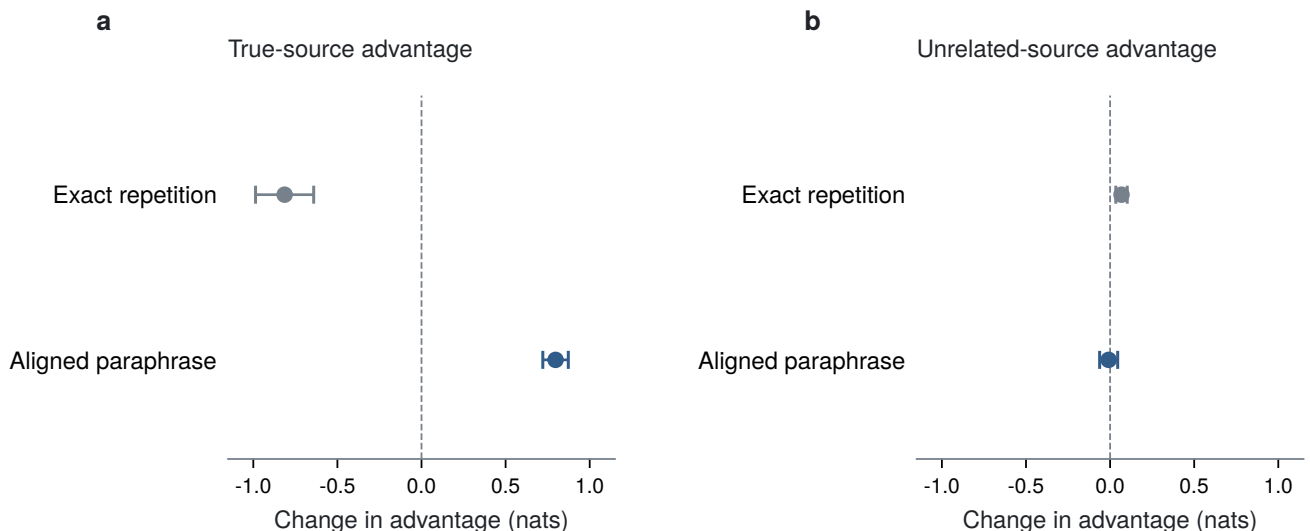


Figure 3 Training relations change the use of source information. (a) True-source advantage; (b) unrelated-source advantage. The horizontal axis shows the change from the reference model in nats, as defined in Equation (4). Targets are compact-restatement tokens absent from the source by tokenizer ID. Points are means over three training seeds; error bars are between-seed standard deviations.

Relative to the reference, exact repetition changes true-source advantage by -0.8138 nats and aligned restatement by $+0.7970$ nats, averaged over three seeds. Standard deviations are 0.1729 and 0.0758 . Unrelated-source advantage changes much less: approximately $+0.0679$ and -0.0085 nats.

Does this difference depend on the text alone, or on the opportunity to use its parts together? Table 7 reports window controls for two seeds. Separating the two passages, while retaining each condition’s complete material, substantially attenuates both the negative repetition effect and the positive restatement effect. The text remains in training but no longer supplies a shared context for a single prediction. Organizing limited experience therefore requires deciding not only which passages to include, but which relationships a prediction can access.

Table 7 Window controls preserving each condition’s paired text. Values are changes in true-source advantage, ΔA_T (nats), averaged over the 80M-, 90M-, and 100M-word checkpoints. Each checkpoint is measured on 2,732 source-nonoverlapping targets. Columns correspond to different training seeds.

Training relation	Window layout	Seed 43022	Seed 43122
Exact repetition	Same window	−0.683	−1.010
Exact repetition	Split windows	+0.009	−0.134
Aligned restatement	Same window	+0.712	+0.824
Aligned restatement	Split windows	+0.116	+0.028

Natural restatements from English and Simple English Wikipedia further reveal target dependence. Table 8 compares 1,200 source pairs separately for targets whose token IDs occur in the source and targets whose IDs do not. Aligned restatement improves true-source advantage when a source token recurs in a different sentence form. Without token overlap, its effect is close to the reference, whereas exact repetition produces a negative effect. Reusing source content in another expression and predicting a source-absent target impose different requirements; training does not transfer indiscriminately between them.

Table 8 Transfer to natural restatements. Entries are changes in true-source advantage, ΔA_T (nats). Targets are averaged within each source pair and target class, then across pairs. Values are means and standard deviations over three training seeds. Each target class contains 1,200 source pairs; overlap is defined by tokenizer ID.

Target class	Exact repetition	Aligned restatement
Target token present in source	+0.109 ± 0.122	+0.238 ± 0.071
Target token absent from source	−0.217 ± 0.029	+0.032 ± 0.040

Transfer is selective: its direction depends on how the practiced relationship corresponds to the relationship required at test time. Same-window versus split-window comparisons identify when that relationship can enter learning; natural-restatement tests identify which targets benefit. Together, these results turn the broad idea of adding related text into a testable principle of experience organization.

This finding connects to work on related-document pretraining, training distributions, and contextual learning [21–23]. Chen et al. directly studied within-window parallel structure by removing it from pretraining text [24]; repetition can also support the formation of particular attention mechanisms [25]. The present experiments distinguish specific effects of repetition and restatement across target classes, test window separation while retaining each condition’s text, and identify transfer settings without the same benefit.

3.2 Familiar Performance and Reuse on Unseen Inputs

After a model learns a relation, can new names or symbols use the same computation? Its *functional reach* is the range of inputs able to use that computation; *functional accessibility* describes whether a particular input can do so. For example, selecting an attribute for a familiar name and performing the same operation for an unseen name are separate tests. Qiushi Engine studied a four-choice relation task with familiar and unseen symbols and intervened directly on internal signals involved in answer selection.

The task specifies a query object and four object–attribute assignments, then asks for the queried object’s attribute. An illustrative instance is “query B; A is red, B is blue, C is green, D is yellow,” whose answer is blue. The experiments use controlled symbols rather than these explanatory words. Tests can retain the rule while replacing the query symbol with one absent from the corresponding relation training. They can also query familiar objects within contexts containing unseen objects. The latter separates difficulty with unfamiliar context from difficulty selecting an attribute using an unfamiliar query.

Full-sequence supervision predicts sequence targets; relation-answer supervision emphasizes the query’s answer position. A static condition fixes the relative weight of these signals, while interleaving alternates relation-answer and full-sequence supervision. Figure 4 reports means for the two training seeds used in the functional-intervention study, 43 and 100, both of which had acquired unseen-symbol transfer during initial relation learning. Initial accuracy is 100.0% on familiar symbols and 87.5% on unseen symbols. Full-sequence continuation yields 87.2% and 40.2%. With a static relation weight, unseen accuracy is 75.5%; with interleaved supervision it is 83.3%, both at 100.0% familiar accuracy. Supervision allocation changes whether new symbols can use the learned relation.

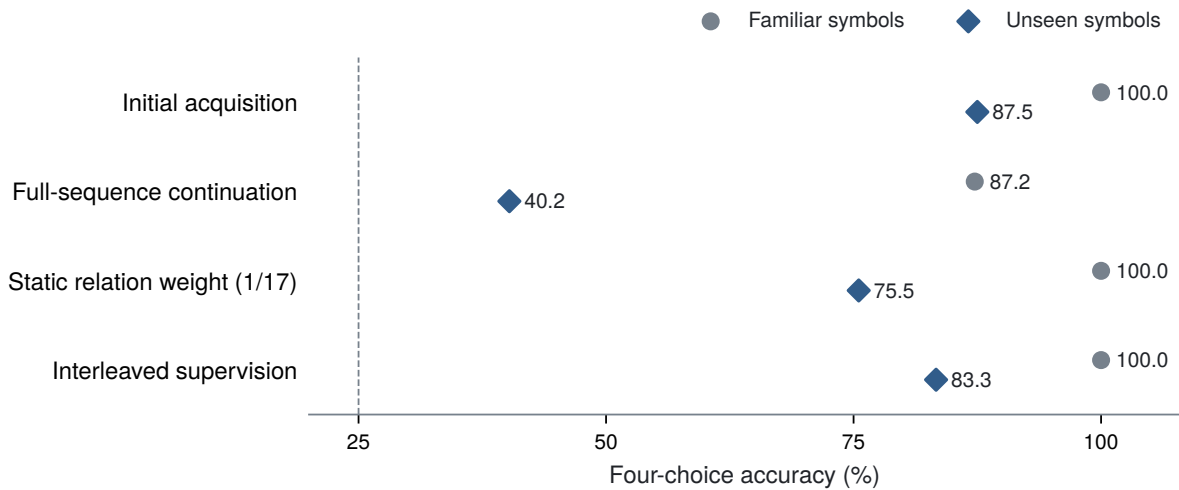


Figure 4 Four-choice accuracy on familiar and unseen symbols. Conditions are initial acquisition, full-sequence continuation, static relation supervision, and interleaved supervision. Points average two training seeds; the dashed line denotes 25% chance accuracy. The static relation weight is 1/17; interleaving alternates relation-answer and full-sequence targets.

Accuracy alone does not identify the computation producing an answer. Each attribute position has an internal vector, or hidden state. In the first layer, the study takes the normalized difference between the mean hidden states at queried and other attribute positions. This selection-related direction is fitted on 1,000 examples from the initial relation model and then held fixed across continuation conditions; each test class uses 384 examples. Interventions change only the projection onto this direction. Centering sets it to the same value at all four positions; zeroing removes it; rotation cyclically transfers its values between positions. All orthogonal components remain unchanged. If removing the signal impairs the answer and moving it redirects the answer, the intervention establishes a functional role in selection.

Zeroing reduces initial unseen accuracy from 87.5% to 25.0%, and interleaved-supervision accuracy from 83.3% to 38.7%. After rotation, a separate measurement asks whether the answer follows the relocated signal. This intervention-target selection rate is distinct from accuracy on the original question.

Table 9 Functional interventions on unseen symbols, in percent, using the same setting as Figure 4. The first three numerical columns measure original-question accuracy. The final column measures selection of the new target indicated by the rotated signal.

Condition	Intact	Centered	Zeroed	Relocated target
Initial acquisition	87.5	26.6	25.0	67.6
Full-sequence continuation	40.2	25.1	31.1	31.4
Static relation weight	75.5	31.3	33.6	48.2
Interleaved supervision	83.3	32.9	38.7	63.9

Measurements through continuation separate loss of the signal from later recovery. Full-sequence training first weakens the answer-selection signal, then restores its use for familiar queries, without comparable recovery for unseen queries. Familiar queries still use the signal in contexts containing unfamiliar objects. The distinction concerns which queries can access a learned computation, rather than a general inability to process unfamiliar context.

An alternative explanation is that the relevant information moves to another linear direction after continuation. The study therefore fits new directions from familiar queries, a single unseen query, or two unseen queries, keeping fitting and testing examples separate. It replaces answer-selection signals along each new direction and measures whether answers move to the designated position. Refitting alone does not alter the model’s predictions; the fitted direction is an intervention tool. Unseen-query redirection remains weaker after full-sequence continuation than after interleaving. These tests do not support the explanation that changing the linear direction suffices to recover the same function. They connect information that can be read from a representation with computation that actually affects behavior [26, 27].

The loss of contextual learning during training and differences between familiar and unseen tokens have prior evidence [28, 29]. Dual Process Learning studies the coexistence of these strategies through weight forgetting. Here the new measurements concern the effects of relation-supervision allocation and the functional consequences of removing or moving a specific signal.

3.3 How Supervision Allocation Changes Relation Learning

Visible source text is only one prerequisite for relation learning. If nearby words already determine the answer, loss can fall without much source use. Supervision allocation determines which predictions are trained and their relative weight. Removing local clues, changing target positions, or changing loss weights can alter the learning problem; relation behavior and ordinary language performance must both be measured.

The controlled experiments compare full-sequence, relation-answer, interleaved, and effective-weight-matched supervision. Relation-misaligned answer targets and a simple reduction in context pressure do not reproduce all the target behavior. The correspondence between the supervised position and the relation being learned is therefore an experimental variable, not merely a matter of total loss or the amount of relational text.

Target-deletion experiments with compact restatements test a related question while preserving the input. A source-absent target is a word absent from its paired source passage; a copied target has a word-level match there. The study removes some prediction losses from

either class. At 100M words, the seven-metric average is 43.896 with full supervision, 43.437 after deleting source-absent targets, and 42.792 after deleting matched copied targets. Target selection changes later performance even with the same input. The deletion counts are close but unequal, while full supervision has more targets, so content and quantity are both involved.

A separate measurement uses the target-deletion models at 20M words and excludes source pairs used in the corresponding training. Across 710 pairs and 974 target events, deleting source-absent supervision rather than matched copied targets raises measured source-absent loss by 0.0796 nats. Resampling by source pair gives a 95% interval of [0.031, 0.125]. Requiring whole-document separation yields an interval containing zero. This early local-prediction test and the 100M-word task evaluation answer different questions: which targets support prediction of specified content, and how the supervision choice affects later task performance. Appendix C preserves both measurements.

Together, these experiments motivate separate tests of whether input supplies the relation, targets require its use, and sufficient learning signal reaches those targets. Separating input corruption from prediction rate has direct precedent [30]. The present work applies that distinction to source–restatement learning and measures the consequences of selecting different supervised positions in otherwise retained text.

3.4 Experience Value Depends on Budget, Displacement, and Learning State

Adding data under a fixed budget necessarily displaces other material. Replacing an equal amount of experience estimates a substitution effect. Let A be added experience, D displaced experience, and B the reference stream. Under fixed training settings, define

$$\Delta_T(A, D; B) = Y_T(B - D + A) - Y_T(B), \quad (5)$$

where Y_T is the evaluation at training amount T . The comparison holds budget, architecture, schedule, and seed fixed. This is an experimental quantity to estimate, not a formula that predicts data value without measurements.

Late-training DeBERTa comparisons distinguish transformed views, expanded sentence coverage, and exact repetition. In another RoBERTa configuration, the view method improves its own training objective without yielding the same broad benefit. Holding the added content fixed while removing child-directed or adult-prose material also changes the result. Experience value depends on displaced information, learning stage, and model state; one early loss or text-distribution distance does not establish a universal ranking.

These studies also uncovered an initialization confound. A seed determines a pseudorandom sequence, but inserting a layer changes how its values are assigned to parameters. The same seed can therefore give shared tensors different initial values. Explicitly copying the common parameters substantially weakened a previously large negative interaction, where interaction means that a data treatment has different effects in different architectures. Actual parameter alignment is essential to that comparison.

3.5 A Design Principle and Independent Findings

Stage II made three conditions separately testable: the information visible during prediction, the targets and weights that receive supervision, and the existing functions to preserve during further training. Natural-text experiments establish effects of relation and window organization;

controlled tasks establish conditions for reuse on new inputs. They jointly motivate coordinated input, supervision, and preservation design.

The corresponding tests are concrete: hold targets fixed while increasing local masking; hold masking fixed while adding targets; change the inputs used for preservation while measuring both acquisition and retained behavior. Stage III implements these choices in real language-model training and uses its controls to refine the design.

Relational anchors, shared representation spaces, identity matching, entity storage, and measurement design also retain independent value. Section 5 explains their questions, constructions, and results. Their scientific value does not depend on inclusion in the second-generation training recipe.

4 Stage III: Principle-Guided Model Improvement

Qiushi Engine returned to the first-generation model with two findings: presenting text does not ensure use of its relationships, and acquiring new behavior does not ensure retention of existing functions. It retained the *Frontier* weights, tokenizer, residual structure, and existing text while changing the training task: which clues remain visible, which tokens must be predicted, and how changes to established predictions are constrained. Comparison with ordinary continuation then tested the practical value of the design.

4.1 From Mechanism Findings to Training Decisions

Relation experiments showed that including the source within the prediction window changes learned source use. Supervision and internal interventions further showed that available information can go unused, and familiar performance need not establish reuse. These findings led to three controllable design choices (Table 10).

Table 10 Connections between scientific findings and the new training design. Each finding motivates a specific intervention and a corresponding comparison.

Finding	Training intervention	Direct test
A visible source need not be used	Retain the source; remove more local clues in the restatement	Hold targets fixed and vary masking; measure source-condition effects
Input conditions and target counts are separate variables	Use dense masking with sparse supervised targets	Compare sparse and dense targets; complete model evaluation
Preservation can also suppress new learning	Preserve parent behavior on ordinary inputs	Compare acquisition, prediction changes, and task scores under ordinary and dense-input preservation

The design developed through revisions. Early explanations emphasized the number of prediction targets. Effects persisted when sparse targets were fixed and masking increased, directing attention to information availability separately from supervision quantity. After developing ordinary-input preservation, the study compared alternative preservation inputs and update magnitudes. These were iterative hypotheses and tests, rather than one complete method fixed before all experiments.

4.2 Separating Visible Inputs from Supervised Targets

Source–restatement pairs come from the first-stage data; no new rewriting is performed in this continuation. Ordinary whole-word masking may leave enough local clues to predict a restatement target without consulting the source. The new method retains the source and masks more candidate words in the restatement, while computing focused prediction loss at only a selected subset. Character spans map candidate words to one or more tokens; these jointly treated units are called *content groups*.

Masking determines what the model can see; supervision determines which prediction errors update its parameters. A position may be masked without contributing its own loss, removing a clue for another target. Dense masking with sparse supervision expands the missing local context without requiring prediction at every masked position.

Let (I, T) denote the combination of input masking and focused targets, with S for sparse and M for dense. *Focused targets* are positions selected specifically for source–restatement prediction; their loss is the focused loss. (S, S) and (M, S) share sparse focused targets but change visible clues. (M, S) and (M, M) share dense masking but change which positions contribute focused loss (Figure 5).

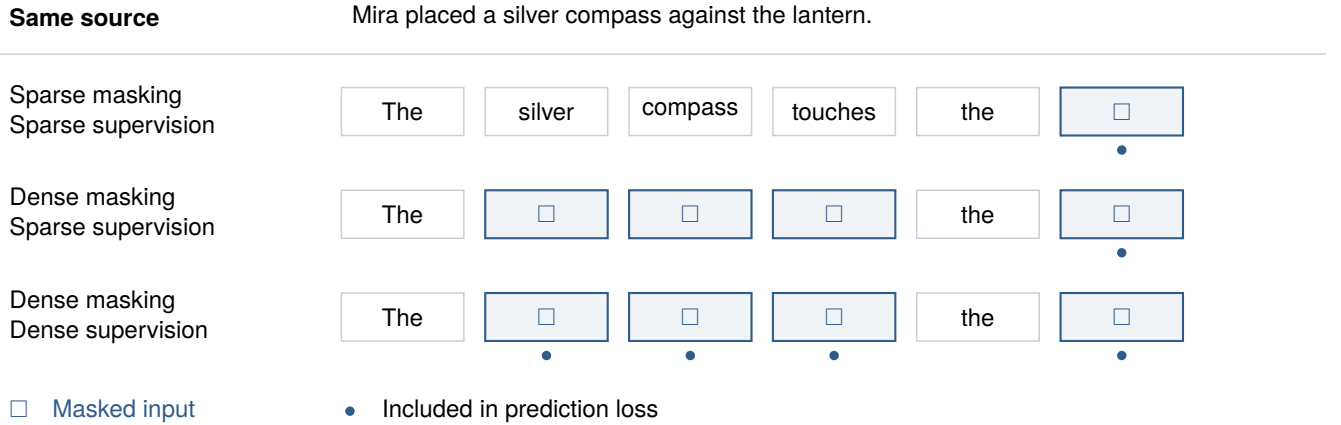


Figure 5 Separating input masking from supervision, illustrated at word level. The first two rows retain targets while changing masking; the final two retain masking while changing targets. Empty boxes denote masked words; dots mark positions contributing prediction loss. Candidate content words are selected while stopwords remain visible. Appendix A gives the actual selection and tokenization rules.

The continuation stream contains 20,475 packed rows and 3,162,742 words. A row is a training input that may contain several segments. The focused relation objective applies to 3,831 rows of existing Qwen rewrites, containing 11,778 paired segments. Other rows retain ordinary language prediction, including Stage I’s compact–restatement and budget–reinvestment material. Dense masking covers 132,283 content groups and 176,607 tokens; sparse supervision selects 21,479 groups and 28,590 tokens. Thus, 148,017 tokens are masked without contributing the focused loss in (M, S) . Sparse and dense are defined relative to the candidate set; Appendix A gives the exact rules.

The acquisition objective combines relation prediction with ordinary language prediction:

$$\mathcal{L}_{\text{acq}} = 0.15 \overline{\text{CE}}_{\text{focus}} + 0.85 \overline{\text{CE}}_{\text{ordinary}}. \quad (6)$$

For each optimizer update, the focused and ordinary losses are each summed over all valid targets in their respective rows, then divided by their own total target-token counts before weighting. This is not an equal-weight average of row or microbatch means. The coefficients weight the two learning signals; they are not fractions of text or tokens. The ordinary objective continues broader language learning while the focused objective trains the relation task.

Input corruption and prediction supervision have been separated in prior work [30]. This study uses that distinction for source–restatement learning: keep the source available, alter local clues on the restatement side, and choose supervised targets separately. The comparisons test both the behavioral effect and the benefit to the first-generation model.

4.3 Preserving Functions on Ordinary Inputs

The new task permits changes to predictions when local clues are missing, while preservation protects behavior under ordinary input conditions. The same existing Qwen paired rows used for relation learning are presented again with ordinary 15% whole-word masking. The updated model and frozen *Frontier* predict the same targets from the same complete rows, with sources retained. KL divergence constrains differences between their probability distributions. Here *ordinary input* denotes the masking procedure, not a separate corpus.

The full objective is

$$\mathcal{L}_{\text{pres}} = \frac{1}{|Q_p|} \sum_{(x,i) \in Q_p} \text{KL}(p_{\text{Frontier}}(\cdot \mid x, i) \parallel p_{\theta}(\cdot \mid x, i)), \quad (7)$$

$$\mathcal{L} = \mathcal{L}_{\text{acq}} + \lambda \mathcal{L}_{\text{pres}}, \quad \lambda = 1, \quad T = 1. \quad (8)$$

Here x is a complete row after ordinary whole-word masking, i is a selected ordinary prediction position, and Q_p collects all preservation targets in the update. KL is computed over the full vocabulary at those positions and averaged over their total count, not over all visible positions. The frozen first-generation model supplies p_{Frontier} ; p_{θ} is the updated model. The coefficient λ sets the preservation weight. Temperature T scales logits before probabilities are computed; $T = 1$ leaves them unscaled. In teacher–student terminology, the teacher is the first-generation model itself.

Forward-pass randomness must be controlled. Dropout randomly disables parts of a neural network during training, so two forward passes can differ even at identical weights. Preservation uses deterministic computation, saving and restoring random-number-generator state so its extra passes do not perturb acquisition. This separates prediction changes due to updated parameters from those introduced by the comparison procedure.

Output-based preservation is directly related to Learning without Forgetting [31]. It also extends the first generation’s existing KL term: the question here is how preservation conditions allow new learning while limiting unnecessary changes on ordinary inputs.

Only the existing increment’s 995,584 parameters are updated, at residual scale 0.75; total parameter count is unchanged. The full method uses 80 updates and adds 517,332 preservation-word presentations beyond acquisition:

$$86,005,295 + 3,162,742 + 517,332 = 89,685,369. \quad (9)$$

Controls without preservation total 89,168,037 words. Additional preservation presentations equal 16.36% of this continuation’s acquisition words and require extra forward computation.

Both remain under the 100M limit. The equal-exposure acquisition comparison and the full method’s combined benefit are reported separately.

4.4 Complete Evaluation Across Two Continuation Seeds

Table 11 reports the common parent and four strategies evaluated on all nine metrics. Seeds 62064 and 62065 vary continuation randomness from the same pretrained model. Ordinary continuation tests whether the new design improves on continued training itself. The (S, S) condition serves a different purpose: mechanism comparisons of input conditions with sparse targets fixed.

Table 11 Complete Overall scores for the shared parent and four continuation strategies. The two seeds share a pretrained starting model. The full method includes additional preservation presentations; all nine component scores are in Appendix B.

Strategy	Seed 62064	Seed 62065	Cumulative words
Parent	42.0240	Same parent	86,005,295
Ordinary continuation	42.0926	42.1159	89,168,037
Dense masking, sparse supervision (M, S)	42.2025	42.1789	89,168,037
Dense masking, dense supervision (M, M)	42.1491	42.1684	89,168,037
Dense masking, sparse supervision + ordinary-input preservation	42.2464	42.2317	89,685,369

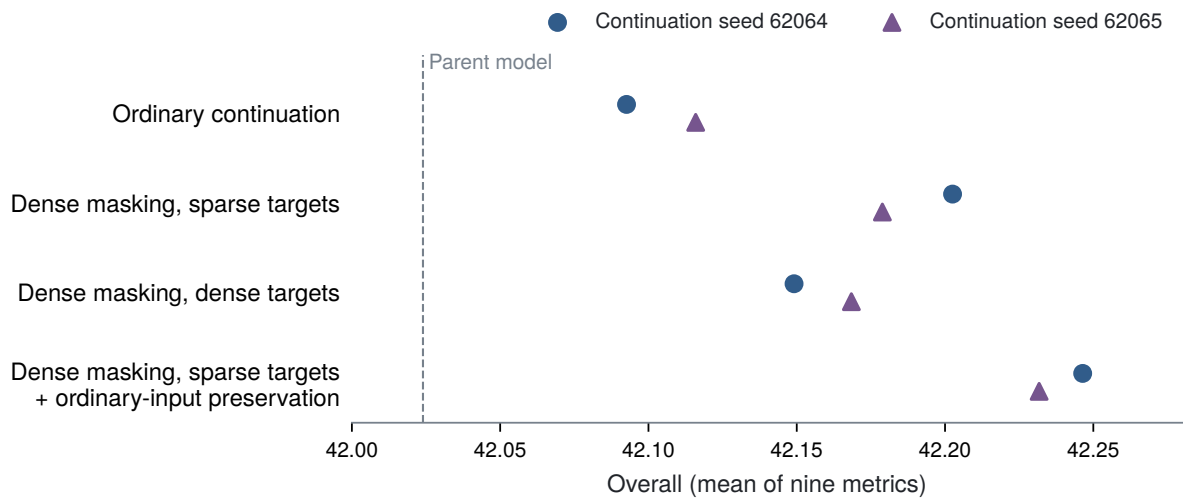


Figure 6 Complete evaluation of continuation strategies from one parent. Circles and triangles represent seeds 62064 and 62065; the dashed line is the parent’s Overall. All candidates have nine-metric evaluations. The full preservation method includes additional ordinary-input presentations.

(M, S) exceeds ordinary continuation by 0.1099 and 0.0630 at the same cumulative word exposure. The full method exceeds ordinary continuation by 0.1538 and 0.1158, and (M, S) by a further 0.0439 and 0.0529. Both dense-target (M, M) models score below the corresponding sparse-target (M, S) models. With the total focused-loss weight fixed at 0.15, distributing supervision over more targets does not improve Overall. This changes both the target set and the relative weight received by an individual target.

The comparisons establish an equal-exposure gain for the acquisition design and a further gain for the combined preservation method. Available (S, S) results comprise screening and mechanism measurements, not the complete nine-metric table. Its masking-specific evidence must therefore be taken from those measurements; the Overall difference between (M, S) and ordinary continuation does not isolate masking alone.

Figure 7 decomposes the full method’s change from the parent. Dividing a component change by nine gives its Overall contribution. GlobalPIQA contributes about 0.165 for both seeds, or approximately 74% and 79% of the total gains. Entity Tracking also improves, while some components decline. The remaining eight components jointly contribute 0.0574 and 0.0428. Net improvement thus combines identifiable gains and trade-offs.

To identify what the method improves beyond continued training, ordinary continuation is the relevant reference. Against that reference, Entity Tracking supplies the largest positive Overall contribution for both seeds, +0.1378 and +0.1511. GlobalPIQA contributes +0.0572 in each case; other components partly offset these gains. Comparing the full method with acquisition-only (M, S) , GlobalPIQA is unchanged: the additional +0.0439 and +0.0529 come from the net changes in other components. Gains from the parent, advantages over ordinary continuation, and the added effect of preservation are distinct comparisons.

An additional check varies downstream fine-tuning rather than continuation. At fine-tuning seeds 42 and 44, the full method’s (Super)GLUE mean exceeds both the parent and (M, S) . This comparison holds the continued models fixed and tests another source of randomness. Table 25 reports all three models under both fine-tuning seeds.

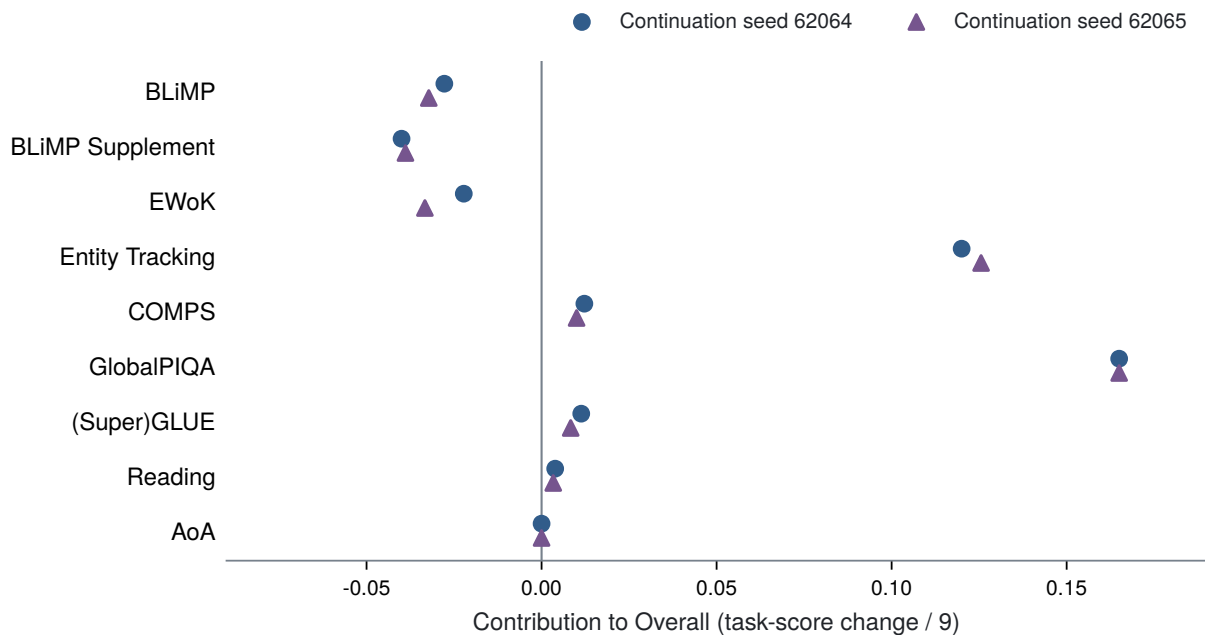


Figure 7 Component gains and costs of the full method relative to the common parent. Each task-score change is divided by nine to give its contribution to Overall. Circles and triangles denote seeds 62064 and 62065. Values are point estimates; both seeds use the same evaluation sets, including the 203-item GlobalPIQA set.

4.5 Acquisition and Preservation Measurements

Three quantities measure new learning on densely masked inputs: correct-target cross-entropy, the correct token’s rank among vocabulary candidates, and KL from the parent to the updated model. Lower CE and a better rank indicate improved prediction; lower KL indicates less output change. The comparison uses 169 previously trained examples and 1,003 shared target tokens. It measures learned behavior on those examples, not generalization to unseen documents.

Table 12 Acquisition and prediction changes on densely masked inputs. All rows use continuation seed 62064 except the explicitly labeled 62065 row. Larger CE and rank improvements relative to the parent are better; KL measures output divergence. The final row applies preservation to densely masked inputs.

Method	CE gain (nats)	Rank gain	Parent KL (nats)
Ordinary continuation	−0.005	0.7	0.001
(M, S) without preservation	0.757	132.8	0.503
Full method, 62064	0.682	128.3	0.353
Full method, 62065	0.679	128.0	0.345
Dense-input preservation	0.159	34.8	0.009

Ordinary-input preservation retains most of the acquisition gain. Dense-input preservation sharply reduces model change but also suppresses improvement on correct targets. A useful preservation method must limit damage to ordinary behavior while allowing the changes required by the new task. Qiushi Engine therefore compared the effective strength of the constraints under different inputs.

With the updated model, parent, and shared target positions fixed, dense masking increases parent entropy by 1.6330 nats, correct-target CE by 1.3155 nats, and teacher-to-student KL by 0.4738 nats. The preservation gradient norm on trainable parameters is 11.835 times its ordinary-input value. A gradient describes the direction and magnitude of parameter change favored by a loss. Here the two gradients have directional similarity 0.7288 but markedly different magnitudes. Equal loss coefficients do not impose equal effective constraints.

Table 13 Preservation constraints at fixed models and shared target positions. The updated model is (M, S) at continuation seed 62064; the reference is the first-generation parent. The first three rows give dense-minus-ordinary input differences; the final row gives the gradient-norm ratio.

Measurement	Result
Parent predictive entropy difference (nats)	+1.6330
Parent target CE difference (nats)	+1.3155
Teacher-to-student KL difference (nats)	+0.4738
Preservation gradient norm: dense/ordinary	11.835

Preservation configurations also differ in their target positions: ordinary masking has 4,762 targets; densely masked positions outside focused supervision have 6,665; their intersection contains the 1,003 positions above. Their effects therefore reflect changes in input type, target set, and effective constraint strength, whose contributions remain partly confounded. All three specify the configuration and must be considered alongside measures of new and retained behavior.

A separate ordinary-input measurement uses 384 complete existing rows and 10,348 targets, retaining sources under ordinary whole-word masking. Parent KL is 0.02449 nats without

preservation and 0.00941 with the full method. CE deterioration relative to the parent falls from 0.0290 to 0.0100 nats. These measurements show reduced change on ordinary inputs; they use a different sample from Table 12, which measures densely masked inputs.

Source interventions use 96 pairs and 186 common targets after the continuation prefix. They are outside this continuation’s trained prefix, but independence from the parent’s pretraining documents has not been established. Correct-source NLL improves slightly, while wrong-source and source-removal conditions incur larger costs (Table 14). The full method reduces some of those costs while retaining correct-source improvement. This measures responses to source changes, separately from generalization of relational reasoning.

Table 14 Source interventions on two models at continuation seed 62064. Values are target NLL differences from the parent, in nats; negative values mean improvement. Every condition uses the same targets.

Source condition	(M, S) alone	Full method
Correct source	−0.024	−0.022
Wrong source	+0.149	+0.099
Restatement only	+0.107	+0.076
Current source erased from full row	+0.210	+0.127
All sources erased from full row	+0.0248	+0.0101

4.6 How the Model Tests Refined the Principle

Stage III established practical value and sharpened the design principle. Increasing masking and increasing target count produced different outcomes, supporting separate control of prediction conditions and supervision. Preservation had to be evaluated through both acquisition and ordinary behavior, rather than proximity to the old model alone. Source interventions further exposed correct-source benefits and wrong-source costs.

The three stages thus form a continuous development of methods: the first supplies a mature model and paired text; the second investigates conditions for information use and capability reuse; the third changes input, supervision, and preservation accordingly and obtains repeated complete-evaluation gains. Model experiments turn the findings into a usable method, while additional controls provide more precise questions for subsequent research.

The new experiments also change the criteria for the next research. Fixed-target masking controls distinguish information availability from target quantity. Gradients measured at common positions show that equal preservation coefficients can impose unequal effective constraints. Qiushi Engine retains the successful method while revising its explanation and recording more informative comparison conditions. This connection between model improvement, method development, and revised scientific understanding underlies the Research RSI discussion in Section 6.4.

5 Research Lineage and Independent Findings

The three-stage program produced both model improvements and independent scientific findings. Compact text, residual architecture, and continued training connect the two model generations through actual inheritance. Studies of relation identification, information retrieval, and measurement developed into complementary research branches. This section explains their questions, their relationship to the main investigation, and the methods and understanding they leave for subsequent work.

The studies span the path from representing text to acquiring capabilities, continuing learning, and measuring outcomes. Table 15 groups their contributions by scientific question. The following sections develop experience substitution, relational anchors, identity matching, and measurement controls. Appendix F preserves the full topic catalog and the status of its conclusions; accompanying materials provide derivations, implementations, experiments, and scientific notes.

Table 15 Research questions and representative outputs across the three stages. Training methods, mechanism findings, and experimental tools each provide a basis for further research.

Research area	Question	Representative outputs
Experience and data	How should limited words be allocated to content, expression, and related passages?	Compact restatements and reinvestment; fixed-budget substitution; relation and window controls.
Representation and architecture	How can sparse supervision identify a relation and make it usable by new inputs?	Character/subword comparisons; zero-initialized residual increments; anchors, shared representations, and identity matching.
Learning signals and capabilities	Which predictions train relation use, and which computations affect answers?	Separate input and target design; supervision allocation; familiar/unseen tests and internal interventions.
Optimization and dynamics	How does experience value change during learning, and how can new and old capabilities coexist?	Early/late trajectories; curriculum and optimizer comparisons; relation dosage; incremental learning and preservation.
Measurement and computation	Which controls make comparisons interpretable and implementations faithful?	Shared initialization; source exclusion and item-level analysis; complete model loading, exposure accounting, and gradient checks.

5.1 Model Inheritance and the Development of Scientific Questions

Compact restatement began as a way to improve a model within its word budget; it subsequently supplied controlled material for comparing expression relationships. Residual incremental learning began as a way to continue training a mature model; it subsequently enabled comparisons of objectives and preservation with a common frozen base. A successful method can thus leave both a model and a new experimental instrument. Across Stages I and II, the objects of research remained available while explanations became more precise.

The two main mechanism branches provide complementary evidence. Natural-text experiments examine how practiced relationships shape source use. Controlled binding tasks examine how queries select attributes and whether new symbols can still use the learned computation. Different tasks and models address a shared design problem: what information a prediction can

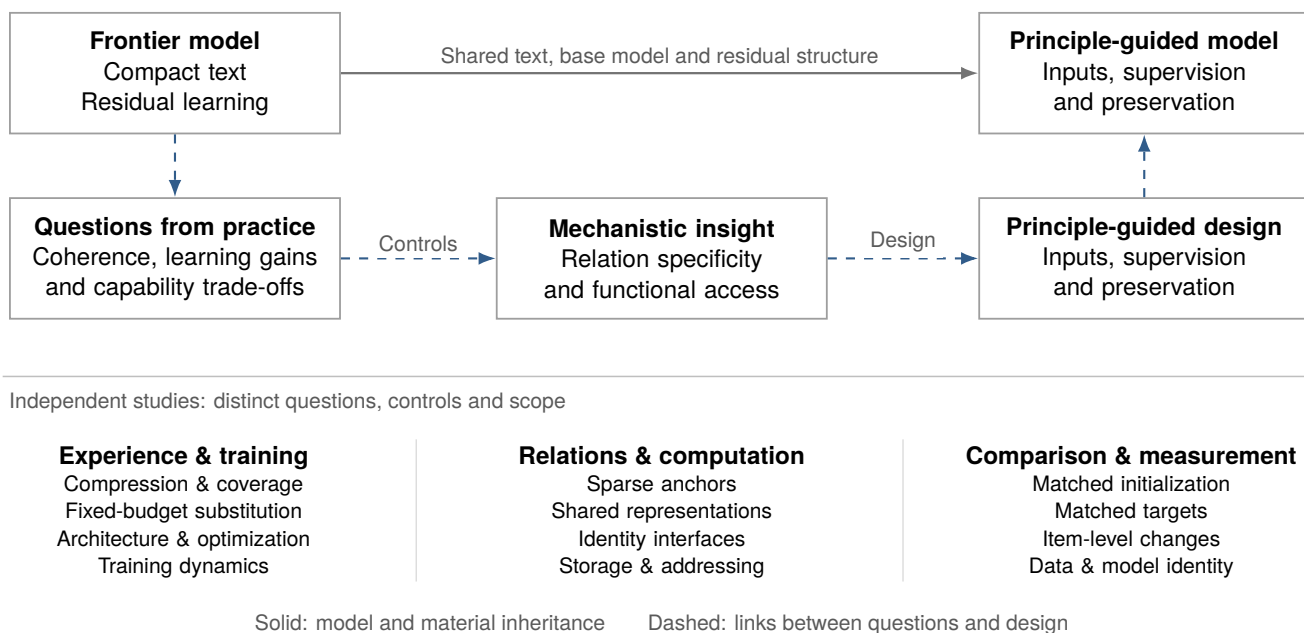


Figure 8 Model inheritance and research branches. Solid lines denote inheritance of first-generation training text, pretrained base, and residual structure. Dashed lines connect practical questions, mechanism studies, and subsequent training design. Independent findings concern experience and training, relations and computation, and comparison and measurement.

access, which targets receive supervision, and which established functions should survive further learning.

Stage III implements these choices by masking more of the restatement, retaining sparse targets, and constraining prediction changes on ordinarily masked inputs. Complete evaluation establishes the method’s practical benefit. Further controls distinguish target count, input conditions, and effective constraint strength. Figure 8 separates actual inheritance of model and data from the way mechanism findings informed the design.

5.2 Experience Compression, Substitution, and Learning Stage

Data processing under a fixed budget can change the expression of existing content or allow more content into training. Compact restatement combines both. To understand their effects, the study treats what is added and what is displaced as separate experimental conditions. This fixed-budget substitution design applies to corpus construction, data mixing, and curricula, beyond the particular restatement method.

At 80M, 90M, and 100M words in one DeBERTa configuration, the study compares compact restatement, different sentences from the same source range, and near-exact repetition. Averaging late results over those checkpoints and five metrics gives differences from the reference of +0.3853, +0.3350, and +0.0343, respectively. The five metrics are BLiMP, Supplement, EWoK, COMPS, and Reading. The different-sentence condition broadens sentence coverage within the specified source range, rather than establishing an expansion of document coverage. Both varied expression and expanded content improve this late-training comparison, while repetition has a smaller effect.

Displaced material matters too. With an added FineWeb block held fixed, replacing child-directed/spoken material rather than adult prose produces a four-metric score 0.6125 points

lower at 100M words. The respective changes from the reference are +0.0825 and +0.6950 on BLiMP, Supplement, EWoK, and COMPS. The added text is identical; different opportunity costs produce different net gains.

In a RoBERTa configuration, restatement improves its own training loss but changes the late five-metric score by -0.6873 . The usefulness of the treatment depends on model configuration and learning stage. Text statistics, such as word-frequency distributions, can help propose candidates; their value is tested through the corresponding task measurements.

Other compression studies distinguish lexical coverage from preserved relationships. Extractive compact views, fluent connecting text with controlled vocabulary, and rule-based rewriting respectively seek broader content coverage, better connections between expressions, or reduced reliance on external generation. Short comparisons show local gains and task costs without the same completeness of final-model evaluation. Their reusable contribution is to make compression quality inspectable: which words, correspondences, and prediction clues survive, and where the released word budget is spent.

5.3 Relational Anchors, Shared Representations, and Identifiability

Some learning problems lack enough information to determine an answer, regardless of training time. Knowing that several objects belong to the same or opposite binary classes identifies their grouping but not which group should be called positive. A few absolute labels can resolve this ambiguity. We call such labels *anchors*.

The study implements this question in a binary relation task. Let an event’s orientation be $z_i \in \{-1, +1\}$ and a comparison between events be $r_{ij} = z_i z_j$. A value of $+1$ denotes agreement and -1 opposition. For connected events, flipping every orientation leaves the pairwise comparisons unchanged:

$$(z_i, z_j) \mapsto (-z_i, -z_j), \quad (-z_i)(-z_j) = z_i z_j. \quad (10)$$

An absolute state label can resolve the remaining orientation ambiguity within this finite hypothesis class. Sparse anchors therefore supply missing absolute information without relabeling every relation.

The model shares input representations and sequence processing between state judgments and relational comparisons, with separate output heads. Embeddings map symbols to vectors; a gated recurrent unit (GRU) processes the sequence. Across three training seeds, reversing a few state anchors reverses indirectly supervised orientations: accuracy against the original absolute labels changes from 1.0 to 0.0. Relative same/opposite judgments remain completely correct. Absolute orientation follows the anchors while relative structure is retained.

The combination of change and invariance identifies what the anchors convey through the shared representation. A matched model with separate representations fits some local tasks but does not show the same propagation. Sharing becomes a testable condition for transferring the anchor information, rather than a general architectural preference.

This construction identifies sufficient information for a specified relational structure and shows how shared representations transmit that information. It relates to work on binding objects, properties, and roles in language models [32, 33]. Candidate objects and relation rules are provided here. Ordinary text additionally requires discovering the objects and interpreting their relations, so the one-anchor result does not directly generalize to all language learning.

Table 16 Anchor reversal in the binary relation task, across three training seeds. The two measurements distinguish absolute orientation from relative structure. Accuracy is on a 0–1 scale.

Measurement	Original anchors	Reversed anchors
Indirectly supervised relations, original labels	1.0	0.0
Relative comparisons between unseen relations	1.0	1.0

Matching names to objects is another prerequisite. A model able to compute a relation still needs to associate its input name with the correct object; we call this mapping the *identity interface*. Character-level experiments supply a small amount of same-letter supervision to align query and candidate representations. In the construction covering all base letters, matching accuracy rises from 0 to 1 and restores relational responses for events without direct supervision. Partial alphabet or training-name coverage leaves new names containing uncovered characters difficult.

New names in this experiment recombine learned characters. Character positions are aligned, candidate objects are provided, and the matcher is frozen after training. Hard assignment selects one candidate instead of mixing candidate representations. Under these conditions, base-character coverage supports new name combinations. Aliases, pronouns, and unrestricted reference remain distinct problems. The experiment separates learning the relation from identifying the object to which it should apply.

5.4 Entity Storage, Retrieval, and Semantic Addressing

Explicit memory stores states at readable and writable locations. Language use additionally requires identifying an object in text, writing the new state to the correct location, replacing outdated content, and retrieving the state required by a query. *Addressing* selects a read or write location; semantic addressing makes that selection from the meaning of the text. Qiushi Engine separates storage, retrieval, and addressing to determine what each component has learned.

Experiments first provide correct read and write locations—the gold-address condition. A slot can be understood as a numbered storage cell. Renumbering reads and writes consistently should preserve the result; exchanging only one location should change the retrieved content. These interventions distinguish invariance to arbitrary numbering from dependence on the information actually read.

With retrieved content connected to language prediction, the three-seed mean answer log-odds gain under correct addressing is 1.2056, the targeted slot-swap effect is 2.0513, and paired accuracy is 0.0472. Log-odds compares the relative probability of a target and a contrast answer; paired accuracy requires both members of a corresponding test pair to be correct. Memory affects answer probabilities, while reliable paired relational behavior remains difficult.

A subsequent design adds memory through $h + gP(m)$, where h is the original representation, m the retrieved content, P a projection to matching dimensionality, and g a gate controlling its contribution. Training also uses a lower learning rate and longer duration. Paired accuracy rises to 0.1375 and the slot-swap effect to 3.087. These are joint changes to the connection and training configuration, not an isolated estimate of the gate’s contribution.

The natural-text difficulty motivated a selection–retrieval decomposition. A reader is first trained to retrieve content from a given location and frozen. A separate selector then uses a soft-max over four candidate locations. This distinguishes inability to read the content from selection

Table 17 Distinct components of entity-memory behavior. Conditions test stored-content effects, address recognition, candidate selection, and temporal state selection.

Condition	Observation	Functional interpretation
Gold addresses	Slot swaps change probabilities; paired accuracy remains low	Memory influences prediction, but the complete relation task is difficult.
Addresses inferred from text	90.4% overall token accuracy; zero key-query recall	Easy positions conceal failure on the required queries.
Frozen reader, learned selector	About 0.994–0.995 on lexical and role swaps	Correspondence selection works among provided candidates.
Temporal state selection	About 0.511 on original/updated state selection	Determining which temporal state to retrieve remains difficult.

of the wrong location when the content is readable. Lexical and role swaps are largely successful, whereas choosing an original or updated state from temporal meaning remains difficult.

Task inspection also removed an apparent positive result. In an early small entity task, the correct answer did not vary with the queried object. A constant answer or first-slot shortcut could therefore produce high accuracy. That result was withdrawn as evidence for compositional binding; correcting the corpus did not itself constitute a new training validation. The storage, addressing, and selection results above come from separately completed experiments.

5.5 Optimization, Initialization, and Measurement

Architecture comparisons require a genuinely shared starting point. In an experiment removing position-related attention terms, the same random seed left 53 of 140 shared tensors with different initial values because model construction consumed random numbers in a different order. Explicit parameter copying changed the estimated architecture-by-data interaction, compact views relative to repetition, from -0.6908 to $+0.0679$, with an interval spanning zero. The reduced architecture still had lower absolute performance.

Two questions must therefore be distinguished: whether deleting a component reduces overall capability, and whether that component changes the relative benefit of a data treatment. The first remained supported; the initially large interaction estimate was affected by initialization. Checking tensor names, shapes, and actual starting values, then aligning common parameters explicitly, provides a reusable control.

Optimization studies likewise separated local measurements from task outcomes. They compared the directions of parameter change favored by different objectives, projected conflicting components, added detection tasks, or switched optimizers. Some interventions improved short-run loss or internal measurements without establishing stable full-task gains. More aligned updates are a hypothesis about learning, not a substitute for measuring the resulting capabilities.

Evaluation uses complementary scales. A macro-average weights task scores equally; an item-level comparison tracks whether each question changes from incorrect to correct or vice versa. Unequal task sizes allow a macro-average to rise while the total number of correct answers falls. The study records gained items, lost items, and their difference, keeping nonaccuracy quantities such as Reading separate.

Implementation choices also affect scientific interpretation. Continuous packing requires checking whole words, source–restatement correspondence, and the content actually visible after truncation. Microbatching requires weighting by target count to preserve the intended

average loss. Gradient checkpointing saves memory by recomputing intermediate activations during backpropagation; short comparisons under fixed randomness test numerical consistency. Trainable parameter fraction, final-model training cost, and the cost of searching for the method are recorded as different quantities.

5.6 Preserving the Full Research Contribution

The independent branches leave four kinds of reusable result: training methods that change real models, mechanism experiments identifying conditions of effect, controls that resolve confounds, and exploratory constructions with explicit questions and implementations. Together they supply executable methods, testable explanations, reliable comparison tools, and directions for further work.

Appendix F organizes methods, controls, and corrections by scientific question, distinguishing completed results from untested candidates. Related methods share topic entries so that readers can move from a question to its intervention, finding, and material. Appendix E links the figures, numerical results, model versions, and research records.

6 Discussion

6.1 The Value of Experience and Contextual Dependencies

Limited experience acquires learning value through the information relationships available during prediction and the demands placed on their use. Content, relative position, expression, and prediction targets jointly define a learning opportunity. Separating paired text across windows weakens source effects despite retaining the material. Different effects of repetition and restatement across targets show that practiced relationships shape subsequent information use.

Coverage, compression, and repetition alter different aspects of experience. Broader coverage changes the range of content; compression changes expression and prediction clues; repetition changes frequency. Comparing them requires asking what information the intended capability needs, whether that information reaches the prediction window, and whether the targets train its use. Changing masking and supervision on existing text yields better results than ordinary continuation at equal cumulative exposure, connecting the relation studies to practical training design.

Improvement at equal exposure directly supports more effective learning in this setting. Scaling studies fit relationships between data, compute, and performance [1]; the present work complements them by explaining and testing choices about experience organization, learned computation, and continued training. Performance curves across data amounts and model sizes can further quantify how those benefits scale.

6.2 Coexistence of New Capabilities and Existing Functions

Acquisition requires model change; retention constrains some of that change. The two can be coordinated by their input conditions: permit learning on the new relational prediction task while limiting unnecessary drift on ordinary language inputs. In these experiments, strong preservation on densely masked inputs suppresses acquisition. Ordinary-input preservation retains most of the new prediction gain while reducing loss changes under ordinary masking.

Preservation also concerns the range of inputs able to use a computation. In controlled tasks, familiar accuracy and unseen-symbol relation processing diverge. Changing supervision improves

unseen-symbol behavior, and internal interventions alter the answers. Together with prior work on structural contextual learning [29], these results motivate evaluation of both familiar behavior and new-input access to learned computation.

Ordinary-input preservation implements this concern in real language-model training and further improves complete evaluation under both continuation seeds. The result establishes the practical value of the full configuration. Input conditions, effective constraint strength, update magnitude, and additional presentations remain separately manipulable variables for explaining its components.

6.3 How Qiushi Engine Organizes and Builds on Research

Qiushi Engine is a hierarchical multi-agent research system developed by the team. Its basic architecture and earlier work on a real optical platform are described by Yang et al. [34]. Core research agents divide responsibility for planning, method construction, experimentation, and critical review. Supporting agents provide retrieval, exploratory assistance, and independent checks. A knowledge system organizes literature, data, and methods; a memory system preserves research progress, experimental evidence, and accumulated methods for later use. Team-developed execution management and tool interfaces connect computing and storage infrastructure, scientific environments, and runtime support. The AI Lab skill supplies callable tools for model generation, training, evaluation, and mechanism experiments.

Qiushi Engine led and executed the continuous scientific investigation, from forming questions to integrating results. The work followed the problem: reviewing literature, constructing data, proposing architectures and objectives, implementing experiments, comparing results, studying mechanisms, and selecting the next investigation. Early and late compact-restatement rankings motivated budget- and learning-stage comparisons. Coherent versus shuffled inputs motivated relation studies. Familiar/unseen performance separation motivated supervision and preservation experiments. Masking effects at fixed targets then revised an early target-count explanation.

Continuity lay in the joint development of methods and scientific understanding. Compact restatements were both a training treatment and controllable material for comparing expression relationships. Residual increments were both a model improvement and an instrument for fixing the base while comparing objectives. Training and evaluation implementations were reused; explanations of why the methods worked were revised. Stage III could therefore build on the first model and data while changing the interaction between input, supervision, and preservation.

These connections show the value of sustained autonomous research: earlier findings change subsequent decisions, while earlier methods support new experiments. Qiushi Engine connects method construction, experimental implementation, analysis, and revision, making a model advance the starting point for further understanding and design.

Idea generation, iterative experimentation, analysis, and writing have been studied in autonomous research systems [35]. The present program supplies a three-stage case in data-efficient language learning: frontier results become scientific questions, and the resulting understanding changes later model training. Its open research materials support further study of how accumulated knowledge affects the quality and efficiency of autonomous science.

6.4 Research RSI: Recursive Self-Improvement of the Research Process

Long-horizon research accumulates more than the best model found so far. Judgments about data value, explanations of learning mechanisms, reusable methods, and experiments that dis-

Table 18 Research observations and subsequent decisions. Findings led to specific questions, controls, mechanism analyses, and training designs.

Observation	New question	Subsequent investigation
Compact views lose early but win late	Does data value depend on budget and learning state?	Fixed-budget substitution, late trajectories, and architecture comparisons.
Coherent and shuffled segments differ	Which relations do the same words practice?	Repetition, restatement, wrong-source, and split-window controls.
Familiar behavior recovers without unseen access	Which inputs recover use of the computation?	Supervision allocation, signal removal, and relocation.
Masking effects persist with sparse targets fixed	Is more supervision necessary for the gain?	Separate input-masking and target-set comparisons.
Ordinary and dense inputs impose unequal preservation strength	How does preservation affect acquisition and retention?	Shared-target gradient comparisons and full-model evaluation.

tinguish competing accounts can all become resources for subsequent work. The three stages show these resources developing together: practical exploration expands the feasible designs, mechanism studies establish their conditions, and later model construction tests whether the resulting understanding is useful.

We use **Research RSI**—Recursive Self-Improvement of the Research Process—to describe this recursive relationship. Qiushi Engine reuses the scientific understanding, method innovations, and experimental experience produced by its own research to change later question selection, experimental strategies, method construction, and evidence assessment. New experiments then test and revise that accumulation. The immediate object of change is the knowledge and methods guiding research, rather than the research agent’s parameters or code. **A research result changes not only what the system has achieved, but how it conducts its next research.**

The changes in this program involve understanding, experimental instruments, and training methods. Early/late differences in compact-restatement performance replace a static data ranking with comparisons conditioned on budget, displaced material, and learning stage. Residual increments develop from a training method into an instrument for studying functional change against a fixed base. Relation and capability-reuse experiments motivate separate input, supervision, and preservation design. These contributions are retained with their conditions, controls, and corrections, not reduced to a list of the best hyperparameters. Tables 18 and 10 connect the changing judgments to the actual training operations.

Research continues after the new model improves. Fixed-target masking comparisons revise an early explanation based on increased supervision. Shared-position gradient measurements show that equal preservation coefficients need not impose equal constraints. Subsequent research can therefore reuse both a successful method and more accurate comparison conditions. Notes, plans, programs, and results preserve how a judgment arose, entered a method, and changed under new evidence. This is the main source of interpretability in the research cycle.

Related systems investigate self-improvement through different objects. STOP applies a program improver to itself [36]; the Darwin Gödel Machine modifies agent code and selects changes by task performance [37]; Reflexion stores verbal feedback for later attempts [38]. Co-Scientist combines hypothesis generation, debate, evolution, and persistent memory in a self-

improving loop, with biomedical validation in collaboration with scientists [39]. Recent reviews also include the research process among possible improvement targets [40]. Qiushi’s contribution is the concrete connection, within one research program, between a real frontier model, the discovery of learning principles, principle-guided redesign, and complete model comparisons, with empirical records of how knowledge and methods develop, combine, and change.

The evidence establishes a recursive research cycle within this BabyLM program: accumulated findings change later designs, model comparisons validate particular methods, and further experiments refine the understanding. It does not establish sustained improvement of general research ability across tasks. That broader question requires independent research goals and comparisons of how knowledge reuse affects experiment selection, research cost, and outcome quality. Here, a frontier advance becomes both a scientific result and knowledge that later research can inherit, test, and develop.

6.5 Reusable Contributions and Further Tests

Three connected contributions emerge: experiments identifying how relation type and window organization shape context use; supervision and internal-intervention studies establishing conditions for capability reuse; and a training method that improves complete model evaluation under repeated continuation. Independent work on compression, anchors, identity matching, and measurement controls broadens the range of methods available for further investigation.

Natural-text controls reveal selective effects of relation organization; controlled tasks explain how new inputs access learned computations; real training tests the benefit of input, supervision, and preservation design. These complementary measurements retain their respective experimental scopes. Independent parent models, different budgets, and document-disjoint tests can establish how widely the connections apply.

Component analysis clarifies the practical improvements. Relative to the parent, GlobalPIQA supplies about 74% and 79% of the full methods’ Overall gains. Relative to ordinary continuation, Entity Tracking supplies the largest positive contribution. The added preservation benefit is the net change in other components, with GlobalPIQA unchanged. Two downstream fine-tuning seeds also show that the full method’s (Super)GLUE advantage is not confined to one fine-tuning run. Each reference answers a different question about the design.

The complete evaluations come from two continuations of one parent, and the full method includes extra preservation presentations. GlobalPIQA has 203 items; both continued models use the same evaluation set, and item-paired uncertainty intervals are not provided here. The reported result is the net Overall improvement and its component changes under the stated controls. Training randomness, evaluation-sample uncertainty, and transfer across parent models are separate quantities.

Counterexamples and corrections are reusable knowledge too. Query-dependence checks, shared-parameter alignment, and text-overlap analysis remove confounds in answer construction, initialization, and sample selection. They provide stronger controls for later studies and identify earlier explanations superseded by subsequent evidence.

Released weights provide experimental starting points. Data construction, target definitions, controls, component scores, and scientific notes explain how the methods formed and how to test them further. Together, the report and repository connect the usable models to their underlying scientific evidence; Appendix E provides direct entry points.

7 Conclusion

Qiushi Engine led and completed three full research stages on BabyLM Strict-Small: frontier-model construction, principle discovery, and principle-guided improvement. Throughout, it formulated questions, developed methods, implemented and analyzed experiments, and synthesized the findings. The stages address one scientific problem: how limited text can teach models to use contextual information relationships while allowing new learning and established language functions to coexist.

The study identifies two important conditions. Exact repetition and aligned restatement selectively shape source use, with effects depending on target relation and prediction window. Recovering familiar performance does not guarantee that unseen inputs can still use a learned computation. The resulting design principle organizes experience around the contextual dependencies required by the target, treats visible information, supervision, and preservation as separate choices, and evaluates new and existing capabilities together.

This principle enters the second-generation training method, which exceeds ordinary continuation on complete evaluation under both continuation seeds from a shared parent. The two public representative models advance from 42.02 to 42.25 Overall. Budget reinvestment, residual increments, relational anchors, identity matching, and measurement controls also leave independent findings, each with its own scientific question and evidence.

For long-horizon autonomous research, the program provides a concrete instance of Research RSI. Qiushi Engine accumulates scientific understanding, technical methods, and experimental experience from its own work, uses them to formulate more precise questions and construct new methods, and tests and revises those designs through practice. Earlier results are inherited and also become objects of study. Effective methods develop, unsuccessful explanations are corrected, and research routes advance through branching and integration.

The two model generations, learning principle, experimental methods, and independent findings form a reusable research foundation. This report explains how they arose and how they connect. The accompanying repository preserves models, code, data construction, complete results, and scientific notes so that others can test the findings and pursue new questions from them.

A Training, measurement, and reproducibility

A.1 Computing environment and experimental support

Research computations used the AI Lab skill and accompanying scientific programs. Here, a skill is a collection of tools available to Qiushi Engine for text generation, training, evaluation, analysis of internal computations, and intervention experiments. CPUs supported text processing, tokenization, data statistics, and result analysis. Two NVIDIA H100 GPUs supported model training and evaluation, teacher-generated text, and accelerated mechanism experiments. Recorded workloads include running `Qwen/Qwen3.5-9B` to produce compact restatements of existing source text, training controlled relation tasks, measuring hidden states and gradients, and intervening on internal signals.

Table 19 Computing platform. GPU use is documented in the research records; CPU, memory, and operating-system specifications were checked on 9 September 2026.

Component	Configuration
GPU	Two NVIDIA H100 GPUs
CPU	Two Intel Xeon Platinum 8473C processors; 104 physical cores and 208 logical threads in total
Memory	Approximately 503 GiB visible to the operating system
Operating system	Ubuntu 22.04.5 LTS, x86_64

Both released model packages record a CPU validation environment comprising Python 3.12, PyTorch 2.11.0 (CUDA 12.8 build), Transformers 4.57.6, Tokenizers 0.22.2, and Safetensors 0.8.0. Dependency records are provided in `models/frontier/ENVIRONMENT.json` and `models/principle_guided/ENVIRONMENT.json`. These describe release validation, not a single environment retroactively assigned to every training, generation, or evaluation run. Original computations retain their own program and dependency records. Environment instructions, training entry points, and data dependencies are collected in `reproducibility/TRAINING.md`.

A.2 Model versions, teachers, and data sources

Both public models have the same architecture and total parameter count. Stage III updates the existing incremental branch. Masked prediction and downstream fine-tuning must load that branch alongside the base network to evaluate the complete model. Input length, padding, and truncation are set by the respective evaluation configurations. Model identity therefore includes the weights, architecture configuration, tokenizer, and corresponding loading implementation.

Shared pretraining used an eight-layer DeBERTa-v2-style architecture with hidden size 480, eight attention heads, and feed-forward size 1,920. A byte-level BPE tokenizer had a vocabulary of 16,384; sequences contained up to 256 tokens. AdamW used an effective batch size of 256, peak learning rate 0.001, linear warmup over the first 6% of training, cosine decay, and weight decay 0.01. The initial residual branch had bottleneck size 128 and scale 1.75. Model configuration, parameter initialization, and training randomness were recorded with seeds 43, 43022, and 43023, respectively. These govern different operations and are not interchangeable descriptions of a single training seed. Table 5 locates first-generation incremental learning and second-generation continuation within the model lineage; further details appear in `models/frontier/TRAINING.md`.

The following links identify the model versions used here. Full revision records are supplied in `results/public_model_revisions.csv`.

Frontier

[Qiushi-Engine-Frontier-Advancement](#).

Principle-guided

[Qiushi-Engine-Principle-Guided-Frontier-Advancement](#).

Three model roles should be distinguished: reasoning models used by the research system, an external teacher that generates rewritten training text, and the frozen parent that supplies preservation targets. Stage III uses *Frontier* as its preservation teacher and reuses previously generated Qwen restatements. Data manifests distinguish 37,594 existing source–restatement pairs containing 1,656,800 words from 12,155 compact pairs containing 423,511 words. Both belong to the ten-million-word corpus pool; no new text is generated for Stage III.

The compact pairs contain 423,511 words of source and restatement text. Nine padding words bring the replacement block in Table 4 to 423,520 words.

Compact restatements were generated with Qwen/Qwen3.5-9B, temperature 0.1, a maximum of 80 generated tokens per call, batch size 64, and bfloat16 precision. These settings and source counts are recorded in `models/frontier/DATA_MANIFEST.json`. Generation templates, pair selection, filtering, and packing records accompany the repository. `data/RECONSTRUCTION.md` explains pair construction, compact replacement, selection of continuation text, and position annotations. The original generator was an external program; its invocation settings and input/output format are retained, with dependencies documented in `reproducibility/TRAINING.md`.

Table 20 describes the actual Stage III input. A row is a packed training input and can contain several segments; row counts therefore describe the continuation stream rather than the word composition of the entire corpus. Corpus-wide source word counts appear in the data manifests. Source licenses, rules for generated text, and cumulative exposure accounting apply to data use.

Table 20 Stage III input composition: 20,475 packed rows containing 3,162,742 words. One row can contain several paired segments.

Source	Rows	Source	Rows
Gutenberg	3,635	CHILDES	5,173
Simple Wikipedia	2,063	Compact restatements and budget reinvestment	940
Existing Qwen source–restatement pairs	3,831	BNC spoken	1,108
OpenSubtitles	3,684	Switchboard	41
Total			20,475

A.3 Acquisition and preservation algorithm

Let Q_o denote ordinary prediction targets, Q_f the relation-focused targets, and Q_m the dense masking set, with $Q_f \subseteq Q_m$. For input x and target set Q , mean cross-entropy is

$$\overline{\text{CE}}(x, Q) = \frac{1}{|Q|} \sum_{i \in Q} -\log p_{\theta}(y_i \mid x, i). \quad (11)$$

Here, y_i is the correct token at position i , $p_\theta(y_i | x, i)$ is its predicted probability, and $|Q|$ counts target tokens. Relation inputs in (M, S) are masked over Q_m but supervised only over Q_f . (M, M) uses the same dense input and a larger supervised set. The equation describes one row for clarity. During training, focused and ordinary losses are each summed across all rows in an optimizer update, divided by their respective total target-token counts, and combined using Eq. (6). Microbatches partition computation without changing these denominators.

Existing annotations determine row assignment. Qwen pairs with source–restatement segment positions enter the focused branch; other rows receive ordinary whole-word masking. In particular, the 940 compact-restatement and budget-reinvestment rows in Table 20 belong to the ordinary branch. Stage I compact text thus continues to support ordinary language learning, while the Stage III focused objective acts on another set of existing paired rows. Preservation presents only those Qwen focused rows again, under a new ordinary whole-word mask, and computes KL at the targets selected by that mask.

Content groups are located as character spans within restatement segments and mapped to valid tokens using tokenizer offsets. Candidate words comprise English letters or digits, allowing internal apostrophes. After lowercasing, words shorter than four characters, purely numerical words, and members of a fixed stopword list are excluded. This is a lexical selection rule. Sparse supervision first samples at most 16 groups per row, then independently selects each with probability 0.35. If candidates exist but none is selected, one group is retained.

Dense masking samples from a separate candidate set, by default selecting at most 128 groups using a row-specific random seed. All supervised groups are then added, so the final set can exceed 128. Masks and labels share the same position mapping but are selected separately. Exact stopwords, row-seed construction, and source-segment annotations are provided with the training implementation.

The complete update proceeds as follows.

1. Read complete rows from the fixed stream. Schedule updates by cumulative words, retaining the established source–restatement pairing and token offsets.
2. Construct relation inputs and sets Q_f, Q_m for annotated Qwen rows; construct ordinary inputs and Q_o for other rows. Count each target class across the update.
3. Normalize each loss by its target-token count, combine them using Eq. (6), and accumulate gradients. Only the designated 48 incremental parameter tensors are trainable. An empty target class raises an error rather than silently changing the loss weights.
4. Apply ordinary masking to the same Qwen rows for preservation. Compute frozen-teacher probabilities without gradients and student probabilities at the same targets. Accumulate $\text{KL}(p_{\text{Frontier}} || p_\theta)$, normalized by the total preservation-target count.
5. Disable dropout for preservation while retaining student gradients. Save and restore Python, PyTorch CPU, and CUDA random-number states around the extra forward passes. Restore training mode and perform the optimizer update.
6. Account separately for acquisition words and preservation presentations, and record checkpoint identity and evaluation vectors. Export the complete model after 80 updates.

Word-paced updates accumulate complete rows until the prescribed word count is reached. Variable row lengths mean that row and token counts per update can vary. Table 21 summarizes the continuation configuration, including the final-update learning rate rather than presenting it as a constant throughout training.

Ordinary continuation supervises 697,102 tokens. (M, S) has 592,858 ordinary targets and 28,590 focused targets. Equal cumulative word exposure therefore does not imply equal target

Table 21 Stage III continuation settings. Both runs share the parent, data processing, and method configuration; continuation seeds are 62064 and 62065.

Setting	Value or definition
Parent	Fixed revision of <i>Frontier</i>
Total / trainable parameters	36,458,592 / 995,584
Incremental tensors / scale	48 / 0.75
Vocabulary size	16,384
Maximum input / microbatch	512 tokens / 8 rows
Continuation seeds	62064, 62065
Optimizer updates	80
Target words per update	Approximately 39,533; complete-row accounting
Learning-rate parameter / schedule	5×10^{-5} ; 455-update schedule, offset 101, warmup 10
Weight decay / gradient-norm cap	0.01 / 1.0
Focused / ordinary loss weights	0.15 / 0.85, each averaged over its own targets
Preservation weight / temperature	1.0 / 1.0; teacher-to-student KL
Preservation input	Complete rows with ordinary 15% whole-word masking and source retained
Acquisition / preservation words	3,162,742 / 517,332
Final-update learning rate (62064)	3.4055×10^{-5} ; a single-update observation

counts or loss allocation; this comparison evaluates a training design rather than an isolated masking-rate change.

Input lengths follow the experiment: main-model pretraining uses 256 tokens, whereas final continuation and the gradient comparison in Table 13 use a 512-token limit. The learning-rate offset continues an existing schedule; the 80 updates do not restart a new schedule. Executed settings are provided in `experiments/configs/executed_stage3.json`, with environments and commands in `reproducibility/TRAINING.md`. Each control retains its own target selection, sampling, and preservation settings.

A.4 Evaluation aggregation, randomness, and trajectories

Every complete-model evaluation interface includes the incremental branch. Zero-shot masked prediction, Reading, downstream fine-tuning, and AoA produce their respective outputs before aggregation by Eq. (2). Local nine-metric comparisons preserve each run’s scoring record under a common aggregation procedure. Public leaderboard tables preserve the displayed values. Four decimal places distinguish close local scores; uncertainty is assessed separately for each measurement.

Scoring is referenced to the official evaluator version cited in BabyLM Organizing Team [11]; the AoA estimator corresponds to revision `6f825c2`. The repository retains the scoring programs actually used and the loading adaptations required by the complete incremental model. `reproducibility/TRAINING.md` links code and execution entry points. Scorer version, model loading, and checkpoint lists jointly define the evaluation configuration; abbreviated labels on a current repository homepage do not replace these definitions.

BLiMP has a reproducible distinction between two scoring records. Local runs score the selected option index, yielding 68.51 and 68.26 for the two generations. Released prediction files retain only the selected answer text. Matching that text against the correct answer yields 68.52 and 68.27, consistent with the public display. Among 59,875 items across 67 subtasks,

seven have identical text in both options, in `passive_1` and `principle_A_case_2`. Three such items for *Frontier* and five for *Principle-guided* receive different scores under the two procedures, accounting for the differences. Released predictions themselves match the corresponding original run files. `results/blimp_scoring_comparison.csv` summarizes the comparison. Original values are retained, and all local training controls use the same scoring procedure.

Macro-averaged task scores and net item changes measure different quantities. For correctness indicators $c_i^{(0)}, c_i^{(1)}$, the net change in correct answers is $\Delta N = \sum_i (c_i^{(1)} - c_i^{(0)})$. Unequal subtask sizes can make this quantity and the macro-average change have opposite signs. Non-accuracy measures such as Reading are not included in item counts. Such a difference occurred between incremental-scale choices in Stage I; it does not describe an item-level comparison of the two official model generations.

Uncertainty follows the sampling unit. Relation experiments report standard deviations across training seeds. Source-isolated loss comparisons resample text pairs, keeping a pair’s targets together. Final-model results report two continuations from one parent. Paired item-level intervals for GlobalPIQA are not provided with this report.

AoA evaluates vocabulary-learning trajectories under the prescribed valid-word and statistical conditions. Both models share 17 earlier checkpoints: one at each million words from 1M to 10M, then one every ten million from 20M to 80M. Adding each model’s own endpoint produces an 18-point trajectory. `CHECKPOINTS.json` in each model directory lists checkpoints and actual cumulative exposure, including their shared history. Lists follow the actual stopping points and do not insert 90M or 100M branches that these models never traversed. Evaluation defaults must be adjusted to the actual trajectory when necessary.

B Complete local evaluation vectors

The following tables list all nine metrics for the shared parent and eight continuation models in the same order. Accompanying CSV files preserve full precision; Appendix E provides file locations. The report builder verifies each nine-metric mean against Overall.

Table 22 Local controls: language form, knowledge, entity tracking, and composition. Sup. denotes BLiMP Supplement. The shared parent appears once.

Strategy	Seed	BLiMP	Sup.	EWoK	Entity	COMPS
Parent	—	68.5100	63.6400	50.0200	28.3200	52.0500
Ordinary continuation	62064	68.4700	63.6300	49.8300	28.1600	52.0000
Ordinary continuation	62065	68.5200	63.6200	49.8100	28.0900	52.0000
Dense masking, sparse supervision (M, S)	62064	68.1200	63.0800	49.9500	29.3900	52.1500
Dense masking, sparse supervision (M, S)	62065	68.0900	63.0900	49.8200	29.2900	52.1400
Dense masking, dense supervision (M, M)	62064	68.0600	63.0400	49.9200	29.3700	52.1500
Dense masking, dense supervision (M, M)	62065	68.0200	63.0900	49.7700	29.4200	52.1500
(M, S) + ordinary-input preservation	62064	68.2600	63.2800	49.8200	29.4000	52.1600
(M, S) + ordinary-input preservation	62065	68.2200	63.2900	49.7200	29.4500	52.1400

Sparse and dense supervision denote (M, S) and (M, M), respectively. The complete strategy adds ordinary-input preservation to (M, S). Quick measurements of (S, S) are not included among complete nine-metric evaluations.

GlobalPIQA is unchanged between (M, S) and the complete strategy. Their Overall difference reflects net changes in other metrics, not simultaneous improvement in every metric.

Table 23 Remaining metrics and complete Overall for the same nine models. Values are shown to four decimal places; source tables retain full precision.

Strategy	Seed	GlobalPIQA	(Super)GLUE	Reading	AoA	Overall
Parent	—	38.5650	68.9457	8.1650	0.0000	42.0240
Ordinary continuation	62064	39.5350	68.9885	8.2200	0.0000	42.0926
Ordinary continuation	62065	39.5350	69.2733	8.1950	0.0000	42.1159
Dense masking, sparse supervision (M, S)	62064	40.0500	68.8878	8.1950	0.0000	42.2025
Dense masking, sparse supervision (M, S)	62065	40.0500	68.9349	8.1950	0.0000	42.1789
Dense masking, dense supervision (M, M)	62064	40.0500	68.5318	8.2200	0.0000	42.1491
Dense masking, dense supervision (M, M)	62065	40.0500	68.8058	8.2100	0.0000	42.1684
(M, S) + ordinary-input preservation	62064	40.0500	69.0477	8.2000	0.0000	42.2464
(M, S) + ordinary-input preservation	62065	40.0500	69.0206	8.1950	0.0000	42.2317

Table 24 Principal Overall differences. The complete strategy includes additional preservation presentations.

Comparison	Seed 62064	Seed 62065
(M, S) minus ordinary continuation	0.1099	0.0630
Complete minus ordinary continuation	0.1538	0.1158
Complete minus (M, S)	0.0439	0.0529
Complete minus parent	0.2224	0.2078

C Additional controls and mechanism results

C.1 Downstream fine-tuning randomness

Table 25 holds fixed the parent and two continuation models trained with seed 62064, then evaluates (Super)GLUE with fine-tuning seeds 42 and 44. It measures variability in downstream training, not additional independent pretraining or continuation. The complete strategy exceeds both the parent and (M, S) under each fine-tuning seed. Main nine-metric results use fine-tuning seed 42; seed 44 is a supplementary comparison and is not combined with other metrics to construct a new Overall. All seven subtask values appear in `results/finetuning_seed_comparison.csv`.

Table 25 (Super)GLUE under two fine-tuning seeds with model weights held fixed. Scores are equal-weight means of the seven primary subtask metrics. Both continuation strategies use continuation seed 62064.

Model	Fine-tuning seed 42	Fine-tuning seed 44
Parent	68.9457	69.3180
Dense masking, sparse supervision (M, S)	68.8878	69.3461
(M, S) + ordinary-input preservation	69.0477	69.6266

C.2 Target deletion and source isolation

A source-absent target is a word absent from its paired source passage, not necessarily a word or fact never encountered by the model. At 20M words of exposure, removing source-absent targets produces higher subword-weighted prediction loss on source-absent content words than

removing whole-word-matched source-present targets. Table 26 distinguishes isolation levels and sampling units.

Table 26 Local loss differences under source isolation (nats). Positive values mean that deleting source-absent supervision is worse on the measured targets than deleting matched-copy supervision. Intervals follow the resampling units of the respective records.

Measurement sample	Target events	Loss difference	95% interval
Pair-disjoint, quality-filtered	974	0.0796	[0.031, 0.125]
Document-disjoint, quality-filtered	71	0.1141	[−0.063, 0.313]
Document-disjoint, all accepted targets	1,012	0.0356	[−0.008, 0.076]

Pair isolation excludes pairs used in the corresponding training data; document isolation also excludes other pairs from the same source document. A loss difference is observed under the former condition. Intervals under the stricter condition include zero. The following table evaluates seven tasks at 100M words, separately from the early local prediction measurement.

Table 27 Seven-metric results of single-seed target-deletion experiments at 100M words. Compact inputs are held fixed; the two deletion conditions remove supervision from 48,105 and 48,387 BPE tokens, respectively. GP denotes GlobalPIQA.

Targets	BLiMP	Sup.	EWoK	Entity	COMPS	GP	Reading	Mean
All	66.87	63.28	53.54	27.75	51.97	35.62	8.240	43.896
Source-absent removed	65.64	63.73	51.51	26.72	51.60	36.665	8.195	43.437
Matched-copy removed	67.32	59.84	51.93	25.67	51.58	35.09	8.115	42.792

Experience replacement, shared initialization, relational anchors, character identity, and entity storage are developed in Section 5. This appendix supplies additional measurements and corrections without repeating those accounts.

C.3 Measurement populations and subsequent corrections

“Unseen” requires a reference set. An example outside the present continuation prefix may already occur in the parent’s training history; distinct text rows may share a source pair or document. A later overlap check found that 6,992 rows previously used for ordinary-loss analysis also occurred in the historical stream. Those measurements are no longer treated as clean evidence of generalization preservation. Further candidate sets likewise require separate checks for exact-row, paired-text, and document overlap.

Results identify the relevant population where they are presented: acquisition on training rows, source pairs outside the current prefix, pair-isolated targets, or document-isolated targets. A correction narrows the claim supported by the affected measurement without automatically invalidating experiments on other identified populations.

KL self-comparisons also require consistent forward-pass randomness. Additional preservation calls must save and restore the acquisition branch’s random-number state. Shared-initialization checks, query-dependence tests, text-overlap analysis, and forward-consistency checks address different experimental confounds. Their programs and records accompany the models for reuse in subsequent comparisons.

D Public leaderboard metrics

Both tables use the same frozen snapshot as Table 3, selecting the two official Qiushi models and eight external submissions without a rank column. Full names and links are provided in `results/leaderboard_comparison.csv`. Publishers are identified by public account, without inferring unreported institutional affiliations.

Table 28 Public submissions: language form, knowledge, entity tracking, and composition. Values retain the precision displayed on the leaderboard.

Publisher / model	BLiMP	Sup.	EWoK	Entity	COMPS
Qiushi Engine / Principle-guided	68.27	63.28	49.82	29.40	52.16
Qiushi Engine / Frontier	68.52	63.64	50.02	28.32	52.05
go76dof / WWM curriculum	67.20	56.01	56.07	28.45	53.57
Serdar404 / RecGPT-10M	73.11	61.73	52.62	16.59	55.43
Wector1 / Wordpiece-24-4	70.20	66.52	52.10	21.82	54.04
gyaudhw / compagg relay	67.85	65.21	53.00	21.47	52.24
gyaudhw / compagg relay (s0)	67.85	65.21	53.00	19.90	52.24
svsatheesh / BabySteps	71.60	63.93	51.94	27.95	53.13
qyxu1994 / instanton-hybrid	72.13	60.86	50.15	19.87	52.96
gyaudhw / synonly (s0)	67.13	62.30	52.28	19.17	52.22

Table 29 Remaining metrics for the same public submissions. Reading and AoA are not ordinary task accuracies; negative values are retained from the snapshot.

Publisher / model	GlobalPIQA	(Super)GLUE	Reading	AoA
Qiushi Engine / Principle-guided	40.05	69.05	8.20	0.00
Qiushi Engine / Frontier	38.56	68.95	8.17	0.00
go76dof / WWM curriculum	39.67	69.79	5.42	0.00
Serdar404 / RecGPT-10M	40.68	66.64	6.92	0.00
Wector1 / Wordpiece-24-4	36.08	69.18	1.84	0.00
gyaudhw / compagg relay	37.59	65.29	5.84	0.00
gyaudhw / compagg relay (s0)	38.65	65.66	5.84	0.00
svsatheesh / BabySteps	36.15	70.44	7.67	-15.10
qyxu1994 / instanton-hybrid	37.14	67.58	6.35	0.00
gyaudhw / synonly (s0)	40.11	66.99	5.82	0.00

Similar Overall scores can conceal different strengths. A model with stronger grammatical judgments may be weaker at entity tracking, and the highest NLP average need not give the highest Overall. Aggregate and component scores serve complementary purposes: Overall records the combined improvement, while individual metrics identify its sources and trade-offs.

E Research repository and use of materials

The accompanying repository organizes models, methods, experiments, and research records by scientific question and research stage. Each reported finding can be followed to its programs, configurations, and results; topic guides also provide derivations, experimental plans, and later revisions. All file paths below are relative to the repository root.

Models. Hugging Face repositories for *Frontier* and *Principle-guided* provide weights, tokenizers, loading implementations, and training and evaluation documentation. Appendix A identifies the versions used here.

Research materials. [Qiushi-Engine-Babylm-Research on GitHub](#) collects both model generations, method implementations, experiment configurations, data construction, complete evaluations, research notes, and editable report sources. Model-package instructions appear in `models/README.md`; scientific materials are organized through the following entry points.

Table 30 Entry points connecting the report to research materials. Method explanations, executed configurations, and original programs serve distinct purposes; principal figure and table data are listed in Table 31.

Research content	Material and purpose
Models and lineage	<code>models/README.md</code> : local weights, tokenizers, custom loading, training records, and loading examples.
Experience construction and incremental learning	<code>methods/compact_views.md</code> , <code>methods/residual_learning.md</code> : Stage I text-budget and model designs.
Relation learning and capability reuse	<code>methods/relation_learning.md</code> , <code>methods/functional_access.md</code> : relations, windows, supervision, and internal interventions.
Principle-guided training and controls	<code>methods/principle_guided_training.md</code> , <code>experiments/configs/executed_stage3.json</code> : complete method, comparisons, and executed configuration.
Training and evaluation programs	<code>experiments/CORE_PROGRAMS.md</code> : original programs; <code>reproducibility/TRAINING.md</code> : environments, arguments, and data dependencies.
Data generation and training inputs	<code>data/RECONSTRUCTION.md</code> : pairing, compression, replacement, packing, and continuation inputs; <code>data/README.md</code> : source licenses.
Questions and design development	<code>research/reading_paths.md</code> : hypotheses, controls, results, and revisions; <code>evidence/research_timeline.md</code> : changes in research decisions.
Research notes and plans	<code>research/notes/README.md</code> : analyses, derivations, and route selection; <code>research/plans/README.md</code> : experimental aims, competing explanations, controls, and decision criteria.
Measurement records and terminology	<code>research/documents/README.md</code> : measurement, data, and technical records; <code>research/terms.md</code> : terms, abbreviations, and experimental conditions used in the materials.
Independent findings and topic catalog	<code>research/materials.md</code> : methods, notes, experiments, and results for 74 topics; <code>research/catalog.md</code> : topic summaries and conclusion status.
Claims and evidence	<code>evidence/claim_evidence_map.md</code> , <code>experiments/index.csv</code> : connections among findings, methods, result tables, and report sections.

English and Chinese PDFs and editable LaTeX projects are in `reports/en/` and `reports/zh/`. Instructions accompany each directory. Numerical figures and tables read the shared `results/` files. From the repository root, `make report-en` rebuilds the English report and `make report` rebuilds the Chinese report. Neither command runs model experiments or calls external models.

Table 31 Principal result data and report build files. Paths are relative to the GitHub repository root.

File	Contents and use
results/training_strategy_comparison.csv	Nine local models, all nine metrics, exact Overall, continuation seeds, and cumulative exposure.
results/leaderboard_comparison.csv	Two representative models and eight external submissions from one snapshot, with links and displayed scores; no rank column.
results/public_model_revisions.csv	Public model repositories and fixed revisions.
results/relation_context_use.csv	Source-advantage changes on compact-restatement targets and standard deviations across three training seeds.
results/relation_window_controls.csv	Same-window and split-window controls, seeds, target counts, and checkpoint aggregation.
results/natural_restatement_transfer.csv	Two target classes in natural restatements and source-advantage changes across three seeds.
results/finetuning_seed_comparison.csv	Three models under two fine-tuning seeds: seven subtasks and the (Super)GLUE mean.
results/blimp_scoring_comparison.csv	BLiMP scoring by selected option index and by stored answer text.
results/interface_reach.csv	Familiar/unseen accuracy and functional-intervention measurements from one controlled setup.
results/state_preservation_readouts.csv	Acquisition and preservation measurements with their distinct sample populations.
results/compact_budget.csv	Compact restatement, source, and budget-reinvestment counts.
results/source_disjoint_target_loss.csv	Pair- and document-isolated target losses and intervals.
results/target_deletion_100m.csv	Seven-metric results for three target-deletion conditions.
results/late_consolidation_controls.csv	Stage I late continuation and incremental controls, update methods, and exposure.
results/private_scale_sweep.csv	Historical measurements used to select the Stage I incremental scale.
results/preservation_gradient_diagnostic.csv	Preservation strength and gradient direction on shared target positions, with measurement counts.
results/research_catalog.tsv	Scientific topics, result status, and reading guides.
reports/zh/render.py	Shared data checks and vector statistical figures.
reports/en/render.py	English tables generated from the same result files.
reports/zh/figures/input_supervision.tex	Editable vector source for masking and supervised positions.
reports/zh/figures/research_lineage.tex	Vector source for model inheritance, method links, and independent branches.
reports/en/build.sh	English LaTeX and bibliography build.

Across 74 topics, the materials retain model methods, mechanism findings, controls, and scientific corrections. Research notes and experimental plans explain how questions arose, why methods changed, and which results informed subsequent work. Models, code, documents, and third-party data remain subject to their respective licenses. Redistribution terms and data acquisition or construction procedures appear in `data/README.md` and the relevant material descriptions.

F Research topics and findings

This catalog organizes the research by scientific question. Each entry identifies the intervention, finding, status, and a direct material link. Reading guides point to related discussion in the report; topic pages locate the specific experimental files and analyses. Neighboring topics may share experiments and materials.

Model method denotes a method used in the corresponding real-model training. *Experimental result* denotes a measurement under specified conditions. *Method or analysis tool* provides an experimental construction, measurement, or analysis procedure. *Conditional result* records an unstable observation or one without complete model evaluation. *Exploratory construction* and *Not run* identify candidates and future directions. *Withdrawn* marks an interpretation invalidated by subsequent checks. Numerical details, comparison conditions, and sample populations are given in the associated discussion and research materials.

Models and representations

Topic	Intervention, finding, and status
R01 Sparse routing and prefix memory	Conditional result. Compare sparse access, capacity, and shared initialization. Local changes did not yield a stable broad advantage; fewer active parameters are not a substitute for comparing capability. (Section 2; Materials: R01.)
R02 Character, subword, and morphological pathways	Conditional result. Distinguish auxiliary word-form structure from capacity effects and lookup controls. Surface-information gains in limited settings have not become a stable broad advantage across seeds. (Section 2; Materials: R02.)
R03 Whole-word and token masking	Experimental result. Compare masking granularity against a fixed training reference, examining paired seeds and individual task scores. Granularity changes task trade-offs, supporting specific training choices but not a uniform ranking across all capabilities. (Section 2; Materials: R03.)
R04 Backbone architecture and batch-size controls	Experimental result. Compare BERT- and DeBERTa-style configurations and check batch-size differences. The late-training advantage of the combined configuration cannot be explained by batch size alone; the contributions of position-related components have not been independently isolated. (Section 2; Materials: R04.)
R05 Recursive autoregressive training	Conditional result. Reproduce the recursive training scheme, complete the corresponding evaluation, and check word coverage. This setting did not match the leading configuration; word-coverage differences cannot explain the entire performance gap. (Section 2; Materials: R05.)
R06 Vocabulary, depth, width, and budget	Experimental result. Compare vocabulary sizes and depth-width configurations, distinguishing prototypes from budget-compliant retraining. Higher tokenization compression ratios did not automatically yield stable gains; the actual training budget must also be accounted for. (Section 2; Materials: R06.)
R07 Frequency gating and fine-grained representations	Exploratory construction. Let coarse-grained word representations draw on fine-grained units and check the zero-gate limit. The limiting implementation behaves as expected, but trained models were not stronger; other gates and seeds remain candidates. (Section 2; Materials: R07.)
R08 Components of a mature training recipe	Experimental result. Examine depth, width, vocabulary, LAMB, and sequence-length and masking curricula. No single component reproduced the full configuration’s performance; interactions among components must also be studied. (Section 2; Materials: R08.)
R09 Gated feed-forward and layer-weighting candidates	Not run. Retain proposed designs for GEGLU, attention gating, and layer weighting. These are candidates, not validated methods, and do not count as empirical model contributions. (Section 2; Materials: R09.)

Experience construction and compression

Topic	Intervention, finding, and status
R10 Aligned reformulation and correspondence	Experimental result. Compare correct and incorrect correspondence, matched sources, repetition, and split windows. Relational organization changes learning, but benefits depend on the task; follow-up studies in the main text further distinguish target types. (Section 3.1; Materials: R10.)
R11 Compact views and budget reinvestment	Model method. Compress the second view and use the freed word budget for more pairs. Experience coverage increased, yielding a practical frontier-model method; the effects of compression and expanded coverage must be distinguished. (Section 2; Materials: R11.)
R12 Adjacency, repetition, and presentation layout	Experimental result. Compare content and adjacency effects, examining block and interleaved presentation. Effects differ across capabilities; tokenizers and window settings must be matched. (Section 3.1; Materials: R12.)
R13 Extractive compression and bridging text	Exploratory construction. Construct bridging texts with balanced coverage, broader coverage, and controlled lexical scope. Word coverage can be checked, but it is not a substitute for semantic relations or gains in the complete model. (Section 5.2; Materials: R13.)
R14 Rule-based rewriting	Conditional result. Compare true pairs, same-entity mismatches, source-only controls, and shuffled controls. Short-run results showed capability trade-offs, not joint gains; visible token counts still need to be matched. (Section 5.2; Materials: R14.)

Learning objectives and supervision allocation

Topic	Intervention, finding, and status
R15 Entity-mention consistency	Conditional result. Compare correct correspondence, incorrect correspondence, and ordinary whole-word masking. Local advantages from correct correspondence did not reliably translate into broad capability gains. (Section 3; Materials: R15.)
R16 Counterfactual and cross-span supervision	Experimental result. Compare true-relation, wrong-relation, and no-relation conditions, checking existing capabilities. An identifiable relation effect is a different conclusion from learning a relation from scratch. (Section 3; Materials: R16.)
R17 State text and directed masking	Conditional result. Construct training texts and target-position controls around state selection. Target behavior can improve while broad language performance degrades, motivating joint tests of new learning and retention. (Section 4; Materials: R17.)
R18 Auxiliary causal objectives	Conditional result. Match shifted targets and gradient norms, comparing batch replacement with auxiliary learning. Similarly directed gradients did not guarantee full-model improvement; local gradient relationships do not establish a general rule for training outcomes. (Section 5.5; Materials: R18.)
R19 Self-sampled replaced-token detection	Conditional result. Add a detection objective and decompose its gradient effects. Short-run aggregate scores showed both gains and costs; overall effectiveness in late training remains unconfirmed. (Section 5.5; Materials: R19.)
R20 Target weighting and supervision coverage	Conditional result. Compare per-word averaging, coverage floors, and source-absent content weighting. Local fit can improve at the cost of broader performance; replacement under an equal target budget still needs separate testing. (Section 3; Materials: R20.)
R21 Relational pivots and counterfactual cues	Conditional result. Match supervision quantity and distinguish relational selection from direct retrieval. If retrieval still solves the task, high scores cannot establish a new computation for state selection. (Section 3; Materials: R21.)

Topic (continued)	Intervention, finding, and status
R22 Gradient exchange and projection	Exploratory construction. Use centered four-condition comparisons, directional projection, and negative examples. Local gradient differences have been observed, but long-term training gains have not been established. (Section 5.5; Materials: R22.)
R23 Coherent-context margin loss	Conditional result. Compare against negatives with disrupted context and examine late-stage continuation. The expected behavioral separation was not obtained; the zero-coefficient comparison was not completed, so the result cannot be fully attributed to a single factor. (Section 5.5; Materials: R23.)

Curricula and learning dynamics

Topic	Intervention, finding, and status
R24 Adaptive masking granularity	Experimental result. Compare data-objective combinations along training trajectories. Early rankings can reverse; the learning stage must be considered when judging a training scheme. (Section 2; Materials: R24.)
R25 Data order and developmental curricula	Experimental result. Use random and developmental orderings of the same content and correct truncation. An effective combined scheme does not establish a universal advantage for curriculum ordering. (Section 2; Materials: R25.)
R26 Context-length caps	Conditional result. Hold paired-example exposure fixed while comparing length caps and evaluation lengths. The observed Overall change is affected by the AoA reference; the seven task families did not improve simultaneously. (Materials: R26.)
R27 Optimizers and mature model states	Experimental result. Compare AdamW, LAMB, optimizer switches at maturity, and trust ratios. Local numerical changes during optimization have not yet been shown to predict late-training model outcomes. (Section 5.5; Materials: R27.)
R28 Checkpoint averaging and restarts	Experimental result. Compare averaging adjacent checkpoints, restarts, and another seed. Averaging did not reliably improve on individual high-scoring models; model selection and methodological attribution must be separated. (Section 2; Materials: R28.)
R29 Relation dose and learning maturity	Experimental result. Compare concentrated, sparse, and role-changing binding experience. Constructed tasks can be learned, but natural-text training with a weak base and sparse exposure did not automatically reproduce the capability; additional budget is listed separately. (Section 3; Materials: R29.)
R30 Continued relation-training candidates	Not run. Retain proposals for continued relation losses, switching between local and full context, and remention masking. These remain routes to test, not achieved results. (Section 3; Materials: R30.)

Residual structures and incremental learning

Topic	Intervention, finding, and status
R31 Zero-initialized bottleneck branch	Model method. Check shared parameters, disabled-branch controls, and training trajectories. The branch preserves the base function at initialization and when disabled; joint training still updates the base. (Section 2; Materials: R31.)
R32 Main and auxiliary path separation	Experimental result. Move from broad to sparse auxiliary learning, including comparisons with the auxiliary branch disabled on the main path. Early gains did not carry over directly to mature models, motivating ordinary coherent replay. (Section 2; Materials: R32.)

Topic (continued)	Intervention, finding, and status
R33 Dedicated increments on a frozen base	Model method. Train only newly added parameters, using ordinary prediction and KL; compare coherent and disrupted inputs. This produced the first-generation model; disabling the branch restores the base, but retention with it enabled still requires evaluation. (Section 2; Materials: R33.)
R34 Model lineage and scale selection	Method or analysis tool. Distinguish continuous-packing branches, the main model, incremental branches, and the inference scale used for release. Actual weight inheritance does not mean that every research route entered the final model in sequence. (Section 5; Materials: R34.)

Entity memory and addressing

Topic	Intervention, finding, and status
R35 Writing and reading with supplied addresses	Experimental result. Compare consistent address permutations, wrong addresses, overwrites, and slot swaps. Stored signals have a functional role; results using gold-standard addresses do not establish natural-language addressing. (Section 5.4; Materials: R35.)
R36 Answer shortcuts in a tiny binding task	Withdrawn. Subsequent checks found that answers did not change with the queried entity. The original high scores no longer support compositional binding; correcting the corpus is not equivalent to verification by retraining. (Section 5.4; Materials: R36.)
R37 Routing from natural text to addresses	Experimental result. Compare raw-text, lexical, recurrent, and discrete routing with correct supplied addresses. High aggregate token accuracy concealed zero recall on critical queries, identifying an addressing bottleneck. (Section 5.4; Materials: R37.)
R38 Pointer and copy initialization	Exploratory construction. Test initialization for small pointer models, embeddings, and copying behavior. Being able to copy does not mean being able to select the correct entity state. (Section 5.4; Materials: R38.)

Selection and readout

Topic	Intervention, finding, and status
R39 Format preadaptation and label alignment	Experimental result. Compare preparatory training with aligned, shuffled, and unrelated labels. Several forms of preparation can help subsequent learning, so the benefit cannot be attributed solely to one particular correspondence signal. (Section 5.4; Materials: R39.)
R40 Frozen readers and within-group selection	Experimental result. Train selection using a four-candidate within-group softmax, measuring lexical, role, and temporal semantics separately. Lexical matching and role swaps were nearly fully successful, but selecting prior versus updated states remained weak. (Section 5.4; Materials: R40.)

Relational structure and identity

Topic	Intervention, finding, and status
R41 Identifiability of relational rules	Experimental result. Vary the initial owner and construct balanced counterexamples. High training scores may reflect an anti-copy shortcut; counterfactuals help identify the intended rule. (Section 5.3; Materials: R41.)

Topic (continued)	Intervention, finding, and status
R42 Binary relation graphs and sparse anchors	Experimental result. Compare shared and separate representations, reverse anchors, and measure whether relative relations are preserved, using multiple training seeds. Absolute orientation can propagate along shared paths while relative relations are preserved; the conclusion is limited to the given hypothesis class. (Section 5.3; Materials: R42.)
R43 Character identity and access to relational structure	Experimental result. Learn matching from letter-equality supervision, then connect it to shared relational structure. Coverage of the base alphabet supports new combinations; candidates and hard assignments are supplied, and edge deletion also changes the training amount. (Section 5.3; Materials: R43.)
R44 Leakage-free graph and program construction	Exploratory construction. Generate neutral, reversed, and program-structured tasks, checking categories and priors. Candidate yield and type balance remain limited; prototypes do not establish general reasoning ability. (Section 5.3; Materials: R44.)

Conditional value of experience

Topic	Intervention, finding, and status
R45 Structural density and representation recovery	Conditional result. Compare structural densities, residualized conditions, and random controls. Recovery of local structural readouts did not guarantee broad gains and must not be conflated with capacity. (Section 5.2; Materials: R45.)
R46 Tail training on shared-source clusters	Conditional result. Compare true, shuffled, repeated, and ordinary experience. Outperforming mismatched controls does not mean outperforming all references; model measurements are incomplete. (Section 5.2; Materials: R46.)
R47 Views, breadth, and repetition under a fixed budget	Experimental result. Replace training experience under a common budget and retain multiple late-stage models. Late-stage gains depend on experience type and architecture; adding distinct sentences is not equivalent to adding documents. (Section 5.2; Materials: R47.)
R48 Register substitution and data-value prediction	Experimental result. Hold added content fixed, vary the displaced register, and compare word distributions and task scores. Opportunity costs differ with the material displaced; results from different measurement settings cannot directly validate the same prediction. (Section 5.2; Materials: R48.)
R49 Selecting explicit relational text	Exploratory construction. Compare explicit relational text with random quality controls and filter natural state text. Qualified candidates are limited; short-run results cannot be treated as a mature large-scale training stream. (Section 5.2; Materials: R49.)

Supervision targets and sources

Topic	Intervention, finding, and status
R50 Deletion of source-absent and copied targets	Experimental result. Keep inputs fixed and nearly match the number of deleted targets, using source-pair-disjoint and document-disjoint splits. Local source-pair results hold, but intervals under stricter document separation cross zero; rankings on the full task set differ. (Section 3; Materials: R50.)
R51 Correct, wrong, and missing sources	Experimental result. Hold target tokens fixed and vary source conditions. Source effects can exist without added supervision and cannot directly be treated as the cause of final-model improvement. (Section 3.1; Materials: R51.)
R52 Coverage, order, and local context	Method or analysis tool. Distinguish source presence, prefix placement, gaps, and relative text positions within windows. Comparisons with incompletely verified model-loading identity illustrate context geometry only; they do not confirm specific mechanism effects. (Section 3.1; Materials: R52.)

Functional mechanisms and learning signals

Topic	Intervention, finding, and status
R53 Ordered composition and dynamic binding	Experimental result. Compare dynamic and static relations, local and final supervision, and examine readouts. Information decodable from representations is not necessarily a computation the model actually performs. (Section 3.2; Materials: R53.)
R54 Contextual hidden states and interventions	Experimental result. Compare consistent prefixes, mismatches, entity substitutions, and activation replacements. Topic information can be present even when state-dependent execution fails; behavioral and functional tests are both needed. (Section 3.2; Materials: R54.)
R55 State updates in microworlds	Method or analysis tool. Control states, actions, contradictions, answer cues, and overwrites. These provide diagnostic tools, not automatic evidence that the same selector has formed in natural text. (Section 3.2; Materials: R55.)
R56 Gradient conflict and learning-signal estimation	Conditional result. Use comparisons with identical materials, across pairs, and with held-out objects. Local correlations have not become stable prospective predictors of learning gains. (Section 5.5; Materials: R56.)
R57 Contrastive prediction and auxiliary readouts	Conditional result. Compare same-source, cross-source, and hard-negative conditions. Fitting or saturating an auxiliary objective is insufficient evidence that new capability has formed. (Section 5.5; Materials: R57.)
R58 Layer-wise readouts and position editing	Conditional result. Use bilinear and ridge-regression readouts, rotated-null controls, and local edits. Local fit does not guarantee held-out transfer; limited editing examples do not support broad mechanistic conclusions. (Section 3.2; Materials: R58.)
R59 Score changes under pair swaps	Method or analysis tool. Check matching conditions including categories, shuffling, and length swaps. Algebraic cancellation in scores does not mean that the model actually used the correspondence. (Section 5.5; Materials: R59.)

Measurement, transfer, and attribution

Topic	Intervention, finding, and status
R60 Shared-parameter initialization	Method or analysis tool. Explicitly copy shared tensors and separately measure architectures' absolute capability and the relative effects of data. The same random seed does not guarantee identical initial values; after correction, evidence for a large interaction was substantially weakened. (Section 5.5; Materials: R60.)
R61 Macro averages and item-level changes	Method or analysis tool. Compute gains, losses, net changes, and complementary items separately. Macro averages and net changes in the number of correct answers can have opposite signs; one aggregate score cannot fully describe capability changes. (Section 5.5; Materials: R61.)
R62 Transfer conditions and representation geometry	Conditional result. Compare seed conditions, principal components, and random controls. Correlations and radial growth have not established capability conservation or a universal law of transfer. (Section 5.5; Materials: R62.)
R63 Complete evaluation and model interfaces	Method or analysis tool. Verify the loading implementation, AoA trajectories, and commonsense-task and Reading scoring. Missing measurements, zero scores, and publicly displayed values are treated separately; complete model identity determines comparability. (Appendix A.4; Materials: R63.)

Systems and computation

Topic	Intervention, finding, and status
R64 Word boundaries, packing, and visible budgets	Method or analysis tool. Check pair integrity, truncation, and actual presentation in continuous windows. Accounted word counts must correspond to visible content; prototype and final tokenizer implementations must not be mixed. (Section 5.5; Materials: R64.)
R65 Microbatches and gradient equivalence	Method or analysis tool. Weight by target count and compare gradients before and after activation recomputation under fixed randomness. These checks validate limited implementation conditions, not exact identity of complete stochastic training trajectories. (Section 5.5; Materials: R65.)
R66 Model reconstruction and cost breakdown	Method or analysis tool. Fix the identities of weights, configuration, and code, separating training from search costs. Same-seed reconstruction, replication across seeds, and compute savings are distinct conclusions. (Appendix A; Materials: R66.)

Core research: extensions and empirical tests

Topic	Intervention, finding, and status
R67 Selectivity in relation learning	Experimental result. Compare source advantages from verbatim repetition and aligned reformulation across three seeds. The effects on compact reformulation targets differ in direction and do not imply that repetition is universally harmful. (Section 3.1; Materials: R67.)
R68 Prediction windows with matched materials	Experimental result. Split repetition pairs and reformulation pairs into separate windows in their respective conditions. The large source effects were substantially attenuated; content is matched between the same-window and split-window versions of each pairing condition. (Section 3.1; Materials: R68.)
R69 Transfer conditions for natural reformulations	Experimental result. Group natural-restatement targets by whether their exact tokenizer ID appears in the source. Aligned restatement improves source use for recurring target tokens; nonrecurring targets show no equally stable improvement over the reference. This distinction is lexical, not a definition of semantic novelty. (Section 3.1; Materials: R69.)
R70 Familiar performance and functional access with unseen symbols	Experimental result. Compare ordinary continuation, statically weighted supervision, and interleaved supervision. Recovery of familiar behavior does not guarantee restored functional reach for unseen symbols. (Section 3.2; Materials: R70.)
R71 Erasing and relocating functional signals	Experimental result. Center, zero, and rotate attribute-position signals, and test refitted directions. The results support the functional role of a specific signal; unsuccessful recovery using linear directions does not rule out all distributed recoding. (Section 3.2; Materials: R71.)
R72 Dense inputs and sparse supervision	Model method. Reuse existing text while controlling masking candidates and focused targets separately. At equal exposure, complete evaluations for both seeds outperformed ordinary continuation; the comparison does not isolate masking as a single factor. (Section 4; Materials: R72.)
R73 Preservation inputs and effective strength	Experimental result. Measure preservation gradients at fixed shared target positions and compare ordinary and dense inputs. With the same coefficient, gradient magnitudes still differ by a factor of approximately 12; the full comparison also changes the set of target positions. (Section 4; Materials: R73.)
R74 Principle-guided complete model	Model method. Recheck ordinary continuation, relation learning, and the complete preservation scheme from the same parent model. The complete scheme improved the nine-task Overall score for both continuation seeds; additional exposure and compute are reported separately. (Section 4.4; Materials: R74.)

References

- [1] Niklas Muennighoff, Alexander M. Rush, Boaz Barak, Teven Le Scao, Aleksandra Piktus, Nouamane Tazi, Sampo Pyysalo, Thomas Wolf, and Colin Raffel. Scaling Data-Constrained Language Models. *Journal of Machine Learning Research*, 26(53):1–66, 2025. URL <http://jmlr.org/papers/v26/24-1000.html>.
- [2] Alex Warstadt, Aaron Mueller, Leshem Choshen, Ethan Wilcox, Chengxu Zhuang, Juan Ciro, Rafael Mosquera, Bhargavi Paranjape, Adina Williams, Tal Linzen, and Ryan Cotterell. Findings of the BabyLM Challenge: Sample-Efficient Pretraining on Developmentally Plausible Corpora. In *Proceedings of the BabyLM Challenge at the 27th Conference on Computational Natural Language Learning*, pages 1–34. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.conll-babylm.1. URL <https://aclanthology.org/2023.conll-babylm.1/>.
- [3] BabyLM Organizing Team. BabyLM Challenge 2026. Official challenge website, 2026. URL <https://babylm.github.io/>. Accessed 8 September 2026.
- [4] Lucas Charpentier, Leshem Choshen, Ryan Cotterell, Mustafa Omer Gul, Michael Y. Hu, Jing Liu, Jaap Jumelet, Tal Linzen, Aaron Mueller, Candace Ross, Raj Sanjay Shah, Alex Warstadt, Ethan Gotlieb Wilcox, and Adina Williams. Findings of the Third BabyLM Challenge: Accelerating Language Modeling Research with Cognitively Plausible Data. In *Proceedings of the First BabyLM Workshop*, pages 399–420. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.babylm-main.28. URL <https://aclanthology.org/2025.babylm-main.28/>.
- [5] Leshem Choshen, Ryan Cotterell, Mustafa Omer Gul, Jaap Jumelet, Tal Linzen, Aaron Mueller, Suchir Salhan, Raj Sanjay Shah, Alex Warstadt, and Ethan Gotlieb Wilcox. BabyLM Turns 4 and Goes Multilingual: Call for Papers for the 2026 BabyLM Workshop. *arXiv preprint arXiv:2602.20092*, 2026. URL <https://arxiv.org/abs/2602.20092>.
- [6] Alex Warstadt, Alicia Parrish, Haokun Liu, Anhad Mohananey, Wei Peng, Sheng-Fu Wang, and Samuel R. Bowman. BLiMP: The Benchmark of Linguistic Minimal Pairs for English. *Transactions of the Association for Computational Linguistics*, 8:377–392, 2020. doi: 10.1162/tac1_a_00321. URL <https://aclanthology.org/2020.tac1-1.25/>.
- [7] Anna A. Ivanova, Aalok Sathe, Benjamin Lipkin, Unnathi U. Kumar, Setayesh Radkani, Thomas H. Clark, Carina Kauf, Jennifer Hu, R. T. Pramod, Gabriel Grand, Vivian C. Paulun, Maria Ryskina, Ekin Akyürek, Ethan G. Wilcox, Nafisa Rashid, Leshem Choshen, Roger Levy, Evelina Fedorenko, Joshua Tenenbaum, and Jacob Andreas. Elements of World Knowledge (EWoK): A Cognition-Inspired Framework for Evaluating Basic World Knowledge in Language Models. *Transactions of the Association for Computational Linguistics*, 13:1245–1270, 2025. doi: 10.1162/tac1.a.38. URL <https://aclanthology.org/2025.tac1-1.57/>.
- [8] Kanishka Misra, Julia Rayz, and Allyson Ettinger. COMPS: Conceptual Minimal Pair Sentences for testing Robust Property Knowledge and its Inheritance in Pre-trained Language Models. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pages 2928–2949. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.eacl-main.213. URL <https://aclanthology.org/2023.eacl-main.213/>.
- [9] Najoung Kim and Sebastian Schuster. Entity Tracking in Language Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3835–3855. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.acl-long.213. URL <https://aclanthology.org/2023.acl-long.213/>.
- [10] Tyler A. Chang, Catherine Arnett, Abdelrahman Sadallah, et al. Global PIQA: Evaluating commonsense reasoning across 100+ languages and cultures, 2025. URL <https://arxiv.org/abs/2510.24081v2>. Version 2, 29 May 2026; first submitted 28 October 2025.
- [11] BabyLM Organizing Team. BabyLM Evaluation 2026. Official evaluation repository; Strict and Strict-Small implementation, 2026. URL <https://github.com/babylm-org/babylm-eval/tree/6f825c291e2c4c78ad33b1935fd64d45f52642dc/strict>. Reference scoring revision 6f825c2; accessed 8 September 2026. Model-specific loading and checkpoint configuration are described in Appendix A.

- [12] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 353–355. Association for Computational Linguistics, 2018. doi: 10.18653/v1/W18-5446. URL <https://aclanthology.org/W18-5446/>.
- [13] Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. SuperGLUE: A Stickier Benchmark for General-Purpose Language Understanding Systems. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/4496bf24afe7fab6f046bf4923da8de6-Paper.pdf.
- [14] Andrea Gregor de Varda, Marco Marelli, and Simona Amenta. Cloze probability, predictability ratings, and computational estimates for 205 English sentences, aligned with existing EEG and reading time data. *Behavior Research Methods*, 56:5190–5213, 2024. doi: 10.3758/s13428-023-02261-8. URL <https://link.springer.com/article/10.3758/s13428-023-02261-8>. First published online 25 October 2023.
- [15] BabyLM Community. BabyLM Leaderboard 2026. Official Hugging Face Space and public source code, 2026. URL <https://huggingface.co/spaces/BabyLM-community/BabyLM-Leaderboard-2026>. Aggregation and sorting source: revision c84d8c1. Public Strict-Small table retrieved 8 September 2026, 12:53 UTC; 129 records. Comparisons use Overall sorting.
- [16] Tyler A. Chang and Benjamin K. Bergen. Word acquisition in neural language models. *Transactions of the Association for Computational Linguistics*, 10:1–16, 2022. doi: 10.1162/tacl_a_00444. URL <https://aclanthology.org/2022.tacl-1.1/>.
- [17] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. DeBERTa: Decoding-enhanced BERT with Disentangled Attention. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=XPZiaotutsD>.
- [18] Microsoft. DeBERTa: Official implementation and DeBERTa-v2 release notes. Official implementation repository, 2021. URL <https://github.com/microsoft/DeBERTa#whats-new-in-v2>. Version 2 release notes dated 3 February 2021; accessed 9 September 2026.
- [19] Thomas Bachlechner, Bodhisattwa Prasad Majumder, Henry Mao, Gary Cottrell, and Julian McAuley. ReZero is all you need: fast convergence at large depth. In *Proceedings of the Thirty-Seventh Conference on Uncertainty in Artificial Intelligence*, volume 161 of *Proceedings of Machine Learning Research*, pages 1352–1361. PMLR, 2021. URL <https://proceedings.mlr.press/v161/bachlechner21a.html>.
- [20] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-Efficient Transfer Learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2790–2799. PMLR, 2019. URL <https://proceedings.mlr.press/v97/houlsby19a.html>.
- [21] Weijia Shi, Sewon Min, Maria Lomeli, Chunting Zhou, Margaret Li, Gergely Szilvasy, Rich James, Xi Victoria Lin, Noah A. Smith, Luke Zettlemoyer, Scott Yih, and Mike Lewis. In-context Pretraining: Language Modeling Beyond Document Boundaries. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=LXVswInH0o>.
- [22] Stephanie Chan, Adam Santoro, Andrew Lampinen, Jane Wang, Aaditya Singh, Pierre Richemond, James L. McClelland, and Felix Hill. Data Distributional Properties Drive Emergent In-Context Learning in Transformers. In *Advances in Neural Information Processing Systems*, volume 35, pages 18878–18891. Curran Associates, Inc., 2022. doi: 10.52202/068431-1371. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/77c6ccacfd9962e2307fc64680fc5ace-Paper-Conference.pdf.
- [23] Akari Haga, Akiyo Fukatsu, Miyu Oba, Arianna Bisazza, and Yohei Oseki. BabyLM Challenge: Exploring the effect of variation sets on language model training efficiency. In *The 2nd BabyLM Challenge at the 28th Conference on Computational Natural Language Learning*, pages 252–261. Association for Computational Linguistics, 2024. URL <https://aclanthology.org/2024.conll-babyLM.23/>.

- [24] Yanda Chen, Chen Zhao, Zhou Yu, Kathleen McKeown, and He He. Parallel structures in pre-training data yield in-context learning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8582–8592. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.465. URL <https://aclanthology.org/2024.acl-long.465/>.
- [25] Nicolas Zucchet, Francesco D’Angelo, Andrew Kyle Lampinen, and Stephanie Chan. The emergence of sparse attention: impact of data distribution and benefits of repetition. In *Advances in Neural Information Processing Systems*, volume 38, pages 39439–39476. Curran Associates, Inc., 2025. doi: 10.52202/085713-1179. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/323b34ddc3b1787a685f4664d8dbb76c-Abstract-Conference.html.
- [26] Atticus Geiger, Hanson Lu, Thomas Icard, and Christopher Potts. Causal Abstractions of Neural Networks. In *Advances in Neural Information Processing Systems*, volume 34, pages 9574–9586. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/file/4f5c422f4d49a5a807eda27434231040-Paper.pdf.
- [27] Fred Zhang and Neel Nanda. Towards Best Practices of Activation Patching in Language Models: Metrics and Methods. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=Hf17y6u9BC>.
- [28] Aaditya Singh, Stephanie Chan, Ted Moskovitz, Erin Grant, Andrew Saxe, and Felix Hill. The Transient Nature of Emergent In-Context Learning in Transformers. In *Advances in Neural Information Processing Systems*, volume 36, pages 27801–27819. Curran Associates, Inc., 2023. doi: 10.52202/075280-1208. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/58692a1701314e09cbd7a5f5f3871cc9-Paper-Conference.pdf.
- [29] Suraj Anand, Michael A. Lepori, Jack Merullo, and Ellie Pavlick. Dual process learning: Controlling the use of in-context vs. in-weights strategies with weight forgetting. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/7db2348b5bfeca620aa7327df815adcc-Abstract-Conference.html.
- [30] Alexander Wettig, Tianyu Gao, Zexuan Zhong, and Danqi Chen. Should you mask 15% in masked language modeling? In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pages 2985–3000. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.eacl-main.217. URL <https://aclanthology.org/2023.eacl-main.217/>.
- [31] Zhizhong Li and Derek Hoiem. Learning without forgetting. In *Computer Vision – ECCV 2016*, volume 9908 of *Lecture Notes in Computer Science*, pages 614–629. Springer, 2016. doi: 10.1007/978-3-319-46493-0_37. URL https://link.springer.com/chapter/10.1007/978-3-319-46493-0_37.
- [32] Jiahai Feng and Jacob Steinhardt. How do Language Models Bind Entities in Context? In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=zb3b6oK077>.
- [33] Qin Dai, Benjamin Heinzerling, and Kentaro Inui. Representational Analysis of Binding in Language Models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 17468–17493. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.emnlp-main.967. URL <https://aclanthology.org/2024.emnlp-main.967/>.
- [34] Shuxing Yang, Fujia Chen, Rui Zhao, Junyao Wu, Yize Wang, Haiyao Luo, Ning Han, Qiaolu Chen, Yuze Hu, Wenhao Li, Mingzhu Li, Hongsheng Chen, and Yihao Yang. End-to-end autonomous scientific discovery on a real optical platform. *arXiv preprint arXiv:2604.27092*, 2026. URL <https://arxiv.org/abs/2604.27092>.
- [35] Chris Lu, Cong Lu, Robert Tjarko Lange, Yutaro Yamada, Shengran Hu, Jakob Foerster, David Ha, and Jeff Clune. Towards end-to-end automation of AI research. *Nature*, 651:914–919, 2026. doi: 10.1038/s41586-026-10265-5. URL <https://www.nature.com/articles/s41586-026-10265-5>.
- [36] Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (STOP): Recursively self-improving code generation. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=46Zgqo4QIU>.
- [37] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Gödel machine: Open-ended evolution of self-improving agents. In *The Fourteenth International Conference on Learning*

- Representations*, 2026. URL <https://openreview.net/forum?id=pUpzQZTvGY>.
- [38] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.
- [39] Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, et al. Accelerating scientific discovery with Co-Scientist. *Nature*, 655:487–496, 2026. doi: 10.1038/s41586-026-10644-y. URL <https://www.nature.com/articles/s41586-026-10644-y>.
- [40] Mingguang Chen, Licheng Wang, and Bo Qu. Recursive self-improvement in AI: From bounded self-refinement to autonomous research loops. *arXiv preprint arXiv:2607.07663*, 2026. URL <https://arxiv.org/abs/2607.07663v2>.